

Federated Heterogeneous Graph Neural Networks for Trade-Based Money Laundering Detection

Submitted by,

Nikhil Vasu

1RV22CS128

Rohith Biradar

1RV22CS164

Suraj Chanaveeragoudra

1RV22CS211

Chapter 1

Introduction to Trade-Based Money Laundering

CHAPTER 1

INTRODUCTION TO TRADE-BASED MONEY LAUNDERING

International trade and digital finance have made global transactions faster, but also harder to monitor. Criminals increasingly exploit this complexity to move illicit value through legitimate trade activity, exposing the limits of conventional Anti-Money Laundering (AML) systems. Trade-Based Money Laundering (TBML) is especially difficult to detect because the suspicious activity is hidden inside normal commercial exchanges. This creates a clear need for intelligent, scalable, and privacy-preserving methods that can uncover hidden transactional relationships in real time.

“For financial institutions, TBML is uniquely challenging because illicit funds are embedded within legitimate trade transactions, leaving traditional AML controls blind to the risks.”

– Dr. Sebastian Hetzler, IMTF Co-CEO [1]

TBML works by blending illicit financial flows into ordinary trade transactions, which reduces the effectiveness of rule-based compliance checks and isolated transaction monitoring.

1.1 Overview of Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) refers to disguising the proceeds of crime through legitimate international trade transactions [2]. Instead of moving money directly, TBML hides illegal value inside import-export activities, invoice settlements, and cross-border trade records. Because these transactions involve multiple intermediaries, institutions, and jurisdictions, conventional AML systems struggle to identify suspicious patterns reliably.

1.1.1 Global Financial Crime Scenario

The growth of cross-border trade has made illicit activity harder to spot in normal financial flows. The United Nations Office on Drugs and Crime (UNODC) estimates that money laundering accounts for nearly 2% to 5% of global GDP, or about \$800 billion to \$2 trillion annually [3]. Europol also reports that authorities intercept less than 1% of

illicit financial flows [4]. These figures show how quickly laundering networks are evolving and why traditional monitoring tools are no longer enough.

As goods, services, and financial assets move across borders, criminal actors can hide suspicious transactions inside legitimate trade activity. Even as banks strengthen Know-Your-Customer (KYC) controls and transaction surveillance, TBML continues to exploit gaps in conventional monitoring.

Table 1.1: Key Statistics Related to Global Trade-Based Financial Crime

Financial Crime Metric	Estimated Value
Global GDP associated with money laundering annually	2% – 5%
Estimated annual TBML economic volume	\$1.6T – \$2.0T
Illicit flows linked to trade misinvoicing in developing economies	87%
Global interception rate of illicit financial flows	<1%
Reduction in false positive alerts using AI-driven monitoring systems	Up to 40%

1.1.2 Trade-Based Money Laundering

TBML allows criminal organizations to move illicit value through trade while avoiding direct cash movement. Common techniques include over-invoicing, under-invoicing, multiple invoicing, false goods descriptions, phantom shipments, and shell entities that distort trade records. Because these actions are spread across real trade operations, they are difficult for regulators and financial institutions to detect.

Global Financial Integrity reports that trade misinvoicing and related TBML activity account for nearly 87% of illicit financial flows from developing economies [5]. Industry estimates also suggest that about \$1.6 trillion to \$2 trillion in illicit value is hidden each year within trade invoices and commercial documents [1]. Together, these findings show how heavily organized crime depends on the international trade system.

1.2 Literature Review

The growth of global finance has led to a large body of AML and TBML research. Earlier work focused on rule-based monitoring, but newer studies increasingly use machine learning, graph analytics, and distributed learning to detect hidden patterns in financial

data. This shift is important because TBML does not usually appear as a single suspicious transfer; instead, it is spread across invoices, counterparties, shipping records, account movements, and commercial relationships. For that reason, recent literature has moved from isolated transaction screening toward methods that can model interactions between entities and uncover structural patterns in the wider financial network.

Jiani Fan, Lwin Khin Shar, Ruichen Zhang, Ziyao Liu, Wenzhuo Yang, Dusit Niyato, and Kwok-Yan Lam [6] surveyed deep learning methods for AML and showed that CNNs, RNNs, and GNNs can improve anomaly detection. Their survey is useful because it presents how learning-based techniques can extract richer patterns than fixed rules alone, especially in environments with large transaction volumes. At the same time, the work remained centered on centralized data processing, which makes it less suitable for institutions that cannot share sensitive records directly. This limitation is especially relevant in TBML scenarios, where data from multiple organizations may be needed to understand the full laundering chain.

Rasmus Ingemann Tuffveson Jensen and Alexandros Iosifidis [7] evaluated standard machine learning models for financial anomaly detection, including Logistic Regression, Random Forests, and Gradient Boosting methods. Their results showed that traditional supervised models can outperform static rule-based approaches when the data is well prepared and labeled. However, the study still treated transactions largely as independent events. In practice, laundering activity is often distributed across connected accounts, counterparties, and repeated trade interactions, so a purely transaction-level view can miss the broader relational structure.

Jie Song, Sijia Zhang, Junghoon Park, Yu Gu, and Ge Yu [8] proposed a dimension expansion framework for learning hidden money laundering activities within transaction networks. The method strengthened transaction representation by combining neighboring account information with contextual interaction features, which is a useful step toward graph-aware fraud detection. The study showed that incorporating relational information can improve the detection of concealed laundering behavior across financial networks. Even so, the approach still relied on manually engineered features, which can reduce flexibility when the structure of the network changes or when the system must operate across distributed institutions.

In another study, Jie Song, Sijia Zhang, Pengyi Zhang, Junghoon Park, Yu Gu, and

Ge Yu [9] introduced a Social Attention Graph Neural Network framework for detecting illicit accounts in blockchain transaction systems. Their work demonstrated the value of attention mechanisms in highlighting the most informative neighbors during graph learning. It also reinforced the idea that graph neural networks can outperform traditional machine learning models in suspicious-account detection. However, the study was focused on blockchain transactions rather than trade documents or invoice-based laundering, so it does not fully address the heterogeneous relationships that appear in TBML cases.

Zhong Li, Xueting Yang, and Changjun Jiang [10] proposed a Multi-View Graph-Based Hierarchical Representation Learning framework for organized money laundering group detection. Their use of multiple graph views helped identify coordinated laundering communities and showed that hierarchical graph representations can capture more than simple pairwise links. The work is valuable because it moves beyond flat feature vectors and models fraud as a collaborative activity. Still, the approach depended on predefined graph relations and centralized repositories, which can limit use in privacy-sensitive environments where institutions must keep their own data local.

Abdul Haluk Batur Gezici and Emre Sefer [11] proposed AMLPD, an unsupervised Anti-Money Laundering detection framework based on Personalized PageRank and graph diffusion. This work is important because it reduces dependence on labeled data, which is often expensive and incomplete in real banking systems. The method was able to capture hidden graph structure in sparse financial networks and demonstrated that unsupervised learning can be practical for anomaly detection. However, the lack of supervised optimization also led to more false positives, which can increase the workload of compliance teams and reduce trust in the system.

Md. Rezaul Karim, Felix Hermsen, Sisay Adugna Chala, Paola De Perthuis, and Avikarsha Mandal [12] investigated scalable semi-supervised graph learning architectures for large-scale financial transaction monitoring. Their evaluation of SkipGCN and EvolveGCN showed that graph learning can scale to bigger transaction datasets while still preserving useful classification performance. This makes the work relevant for real-world fraud monitoring systems that must handle large volumes of records. Even so, the framework assumed centralized graph access and did not directly solve the privacy and regulatory challenges that arise when multiple institutions need to collaborate.

According to a recent paper, we find that AML techniques are slowly departing from

traditional heuristic rule-based methods to graph-based and machine learning techniques. In addition, we pinpoint current AML research challenges such as heterogeneous graph modelling, real-time modelling of changing fraudulent activities, privacy-preserving. Further, existing methods often target either centralized banking or cryptocurrency datasets. Further, they do not have any capability to reflect the complex trading relations in TBML. Additionally, they overlook the financial institution privacy issue and the scalability issue in the distributed heterogeneous trade-finance network. Ultimately, these limitations motivate us to work towards a unified framework, graphFed, which seamlessly integrates heterogeneous graph learning, federated intelligence and real-time detect ability.

1.3 Motivation

The work, “*Federated Heterogeneous Graph Neural Networks for Real-Time Detection of Trade-Based Money Laundering*”, was selected to address the growing complexity of financial crime in modern trade ecosystems. Conventional AML systems often rely on rule-based checks and isolated transaction analysis, which makes them less effective at identifying coordinated laundering patterns spread across connected financial networks. In practice, this leads to more false positives, more manual review, and weaker detection of sophisticated TBML activity.

This project addresses those limitations by combining heterogeneous graph learning with federated intelligence to model transaction relationships while preserving institutional privacy. By using graph attention mechanisms and collaborative training across organizations, the framework supports fraud detection without sharing sensitive transactional data directly. It also adds automated Suspicious Transaction Report (STR) generation and dashboard-based monitoring to support real-time compliance analysis.

1.4 Problem Statement

Existing AML systems rely on rules and isolated transaction analysis, which limits their ability to detect TBML and hidden financial networks. Privacy and regulatory constraints also make cross-institution collaboration difficult. A scalable and privacy-preserving framework is therefore needed to model interconnected financial data and detect suspicious activity in real time.

1.5 Objectives

The objectives of the project are

1. To develop a Federated Heterogeneous Graph Neural Network framework for real-time detection of Trade-Based Money Laundering activities.
2. To model complex transactional relationships among financial entities using heterogeneous graph representation and graph attention mechanisms.
3. To generate Suspicious Transaction Reports (STRs) for identified fraudulent transactions to support financial investigation and compliance processes.
4. To design an interactive dashboard for visualization of fraud predictions, transaction analysis, and real-time monitoring of suspicious activities.

1.6 Brief Methodology of the project

The proposed methodology builds a privacy-preserving graph-based framework for real-time TBML detection. Financial transaction data, trade records, customer information, and entity relationships are first collected and cleaned to create structured analysis datasets. These records are then converted into a heterogeneous graph where accounts, organizations, invoices, and transactions are linked as nodes and edges.

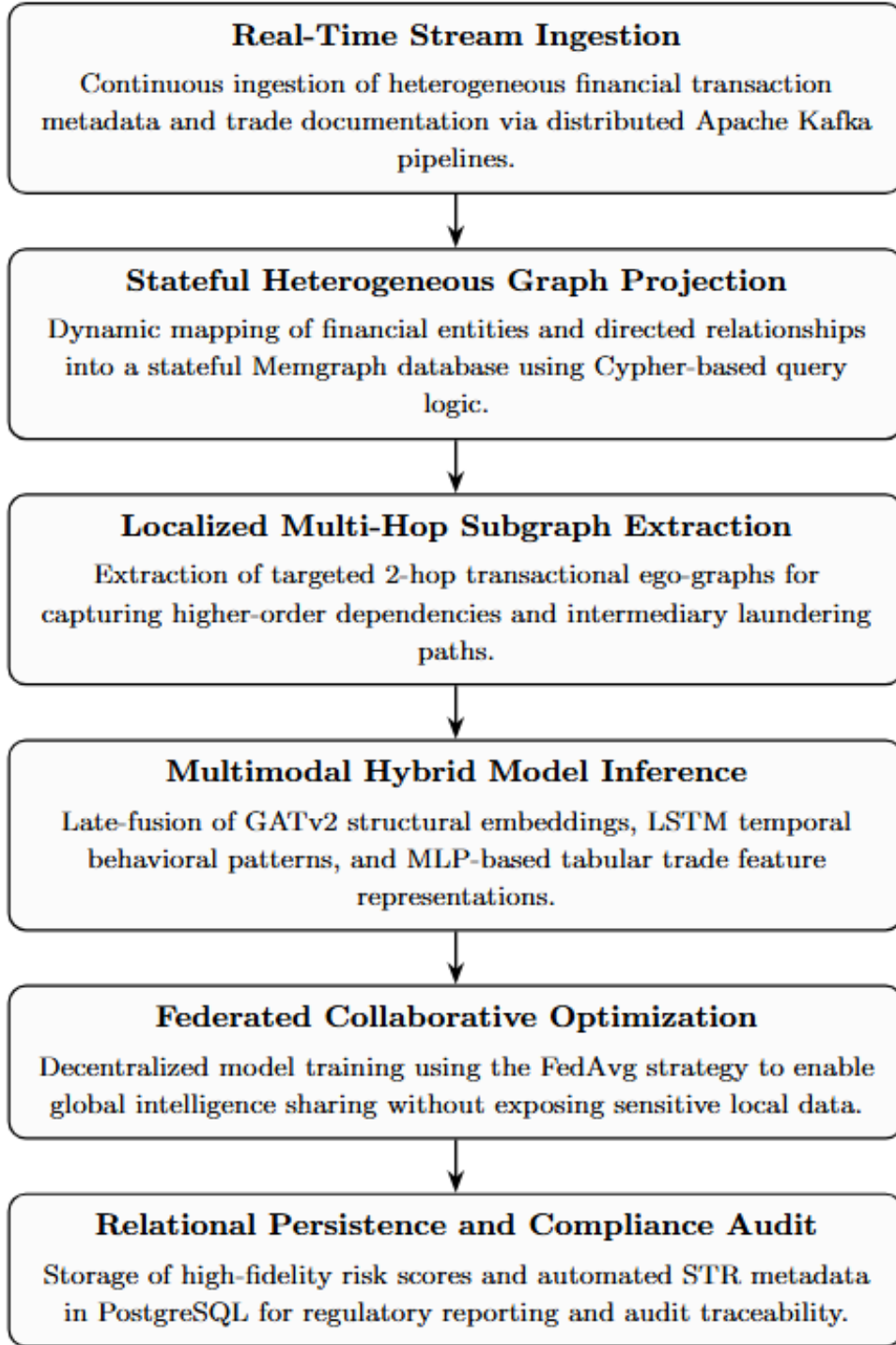


Figure 1.1: Proposed Methodology for TBML Detection

Graph attention mechanisms are used to capture hidden relationships and suspicious behavioral patterns within the financial ecosystem. Federated learning is then added so that institutions can train collaboratively without exposing sensitive local data. The detected suspicious activities are finally used to generate STRs and support real-time

monitoring through an interactive dashboard.

1.7 Assumptions and Constraints of the Work

Assumptions

- The financial transaction records, trade invoices, and customer-related information used for analysis are assumed to be sufficiently structured, consistent, and preprocessed for effective model training and evaluation.
- It is assumed that participating financial institutions maintain similar transactional formats and standardized trade-related attributes required for graph construction and suspicious activity analysis.
- The generated financial datasets are assumed to contain sufficient normal and suspicious transactional patterns necessary for learning hidden behavioral relationships associated with Trade-Based Money Laundering activities.
- It is assumed that heterogeneous graph representations can effectively model relationships among accounts, organizations, transactions, invoices, and trade entities for identifying suspicious financial behavior.

Constraints

- Limited access to real-world Trade-Based Money Laundering datasets due to institutional privacy regulations and confidentiality policies may affect the diversity of transactional data used for evaluation.
- Large-scale graph construction and real-time transaction analysis may increase computational complexity and memory requirements during fraud detection operations.
- The performance of the proposed framework depends significantly on the quality, completeness, and accuracy of the financial data available for model training and testing.
- The proposed framework is validated primarily under controlled academic and experimental conditions and may require additional optimization for large-scale industrial deployment.

1.8 Organization of the Report

This report is organized as follows.

- Chapter 2 presents the theoretical foundations relevant to the proposed Trade-Based Money Laundering (TBML) detection framework, including Anti-Money Laundering concepts, graph-based financial modeling, heterogeneous graph learning, Graph Neural Networks, federated learning, and real-time monitoring technologies.
- Chapter 3 discusses the design of the proposed system by integrating the Software Requirements Specification (SRS), high-level architectural design, and detailed module-level design. The chapter describes the system requirements, architectural components, workflow design, data-flow mechanisms, graph-construction methodology, and the functional design of the major system modules.
- Chapter 4 presents the implementation and testing of the proposed framework. The chapter describes dataset preparation, graph generation, model development, federated learning implementation, backend and dashboard integration, and the testing procedures performed to validate the functionality, reliability, and operational behavior of the system.
- Chapter 5 presents the experimental results and performance evaluation of the proposed framework using various fraud-detection metrics, scalability analysis, federated learning observations, and system-level analytical evaluation.
- Chapter 6 concludes the report by summarizing the major findings and contributions of the proposed work. The chapter also discusses the limitations of the current implementation, learning outcomes, and potential directions for future enhancement.

Chapter 2

Theory and Fundamentals of Project

CHAPTER 2

THEORY AND FUNDAMENTALS OF PROJECT

Building an intelligent framework for Trade-Based Money Laundering detection requires a solid understanding of financial crime analysis, graph theory, and machine learning. This chapter introduces the core ideas behind Trade-Based Money Laundering, Graph Neural Networks, Graph Attention mechanisms, heterogeneous graph representation, and federated learning. It also explains the computational ideas behind relational transaction modeling, privacy-preserving learning, and real-time fraud analytics. Together, these concepts provide the theoretical base for the proposed fraud detection framework.

2.1 Trade-Based Money Laundering

Trade-Based Money Laundering (TBML) is a form of financial crime in which illicit value is moved and concealed through apparently legitimate international trade activity. Unlike traditional laundering methods that rely mainly on direct financial transfers, TBML makes use of import-export systems, trade invoices, shipping documents, and cross-border commercial operations to disguise the movement of illegal funds. As Anti-Money Laundering (AML) regulations and Know Your Customer (KYC) requirements have become stricter, criminal groups have increasingly moved toward trade-based methods that are harder to detect with conventional monitoring systems.

Because global trade is complex and involves many financial entities and transaction relationships, TBML remains difficult to detect for both financial institutions and regulatory agencies. This makes analytical methods that can reveal hidden dependencies and suspicious financial behavior especially important in modern Anti-Money Laundering systems.

2.1.1 Introduction to TBML

Trade-Based Money Laundering primarily involves the manipulation of international trade transactions to illegally transfer value between entities located across different countries. Criminal organizations often misuse trade documentation, invoice values, shipment quantities, and commercial agreements to move illicit funds while making the transactions appear legitimate within global financial systems. Since these laundering activities are embedded within regular import-export operations, identifying suspicious behavior

becomes highly challenging for financial institutions and regulatory agencies.

In many TBML operations, colluding buyers and sellers intentionally manipulate the declared value of traded goods to transfer illegal financial assets across borders. Techniques such as over invoicing, under invoicing, phantom shipments, and multiple invoicing are commonly used to conceal the movement of illicit proceeds. Such laundering mechanisms exploit the complexity and high transactional volume of international trade systems, making traditional rule-based Anti-Money Laundering approaches comparatively ineffective in detecting hidden financial relationships.

Figure ?? illustrates a typical Trade-Based Money Laundering workflow involving collusion between exporters and importers across different jurisdictions [13].



Figure 2.1: Trade-Based Money Laundering through trade misinvoicing and open-account transactions

2.1.2 Challenges in Detecting TBML

Detecting TBML is difficult because modern trade systems are highly connected and constantly changing. Unlike ordinary financial fraud, TBML activity is usually embedded within legitimate business transactions, which makes suspicious behavior harder to spot. Criminal networks often rely on shell companies, intermediaries, multiple jurisdictions,

and layered transactions to move illicit funds while appearing to operate normally.

Traditional AML systems mostly depend on rule-based monitoring and isolated transaction analysis. These methods can detect known suspicious patterns, but they often miss the hidden relationships between entities in large trade networks. At the same time, the growth of cross-border transactions, changing laundering tactics, and restrictions on sharing sensitive data between institutions make TBML detection even more challenging. These conditions create a strong need for scalable and privacy-preserving detection frameworks.

2.2 Algorithms Used

This section presents the major algorithms and deep learning techniques used in the proposed system for traffic prediction and autonomous vehicle navigation. The project combines graph-based learning and sequential modeling approaches to effectively analyze complex traffic patterns and temporal dependencies. Graph Attention Network v2 (GATv2) is utilized to capture spatial relationships between connected road networks through dynamic attention mechanisms, while Long Short-Term Memory (LSTM) is employed to learn temporal traffic variations and long-term sequential dependencies. Together, these algorithms improve traffic prediction accuracy, congestion analysis, and intelligent decision-making within the proposed framework.

2.2.1 Graph Attention Network v2 (GATv2)

Graph Attention Network v2 (GATv2) is a Graph Neural Network (GNN) architecture that uses a dynamic attention mechanism to learn relationships between connected nodes in graph data. It improves feature aggregation by automatically identifying the importance of neighboring nodes. GATv2 is widely used in applications such as traffic prediction, recommendation systems, and autonomous vehicle navigation.

Introduction

Graph Attention Network v2 (GATv2) is an advanced Graph Neural Network (GNN) architecture designed for learning on graph-structured data using a dynamic attention mechanism. It improves upon the original GAT model by overcoming the limitation of static attention and enabling better learning of relationships between connected nodes. GATv2 automatically identifies the importance of neighboring nodes during feature aggregation, making it highly effective for applications such as traffic prediction, recom-

mendation systems, social network analysis, fraud detection, and autonomous vehicle navigation.

Fundamental Concept of GATv2

The primary objective of GATv2 is to generate node embeddings by aggregating information from neighboring nodes using a dynamic attention mechanism. For each node, the model learns which neighboring nodes are important and assigns suitable attention weights before combining their features.

Architecture and Working of GATv2

The overall architecture and workflow of GATv2 are shown in Figure 1.1.

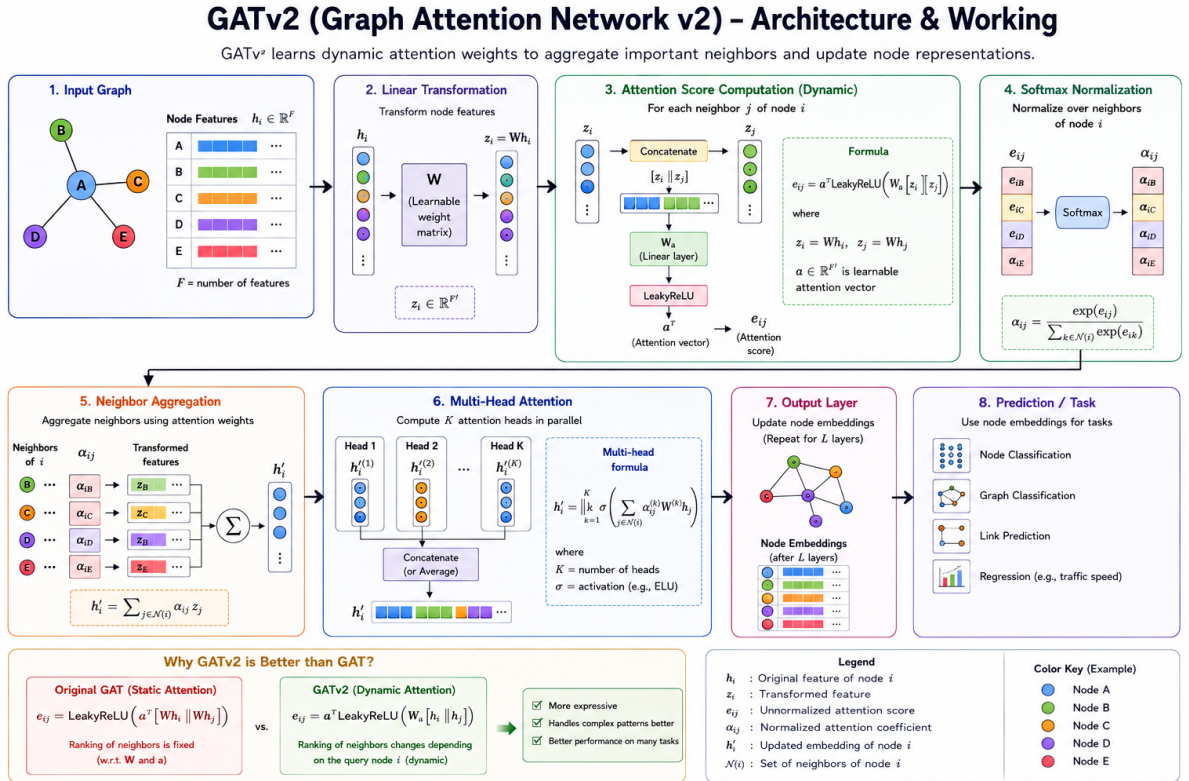


Figure 2.2: GATv2 Architecture and Working

The architecture of GATv2 consists of graph input processing, feature transformation, attention computation, normalization, neighbor aggregation, multi-head attention, and prediction generation.

Step 1: Input Graph: The process begins with a graph containing nodes, edges, and node feature vectors. Each node feature vector is represented as:

$$h_i \in \mathbb{R}^F$$

where h_i denotes the feature vector of node i and F represents the number of input features. In traffic prediction systems, nodes may represent road intersections while edges represent road connectivity.

Step 2: Linear Transformation: The input node features are transformed using a learnable weight matrix:

$$z_i = Wh_i \quad (2.1)$$

where W is the learnable weight matrix and z_i is the transformed feature representation. This transformation improves feature learning before attention computation.

Step 3: Attention Score Computation: For every connected node pair (i, j) , the attention score is computed as:

$$e_{ij} = a^T \text{LeakyReLU}(W[h_i \| h_j]) \quad (2.2)$$

where a is the learnable attention vector and e_{ij} represents the attention score between neighboring nodes.

Step 4: Softmax Normalization: The attention scores are normalized using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})} \quad (2.3)$$

where α_{ij} denotes the normalized attention coefficient and $\mathcal{N}(i)$ represents the neighboring nodes of node i .

Step 5: Neighbor Aggregation: The neighboring node features are aggregated using weighted summation:

$$h'_i = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} Wh_j \quad (2.4)$$

where h'_i is the updated node representation.

Step 6: Multi-Head Attention: GATv2 employs multiple attention heads operating in parallel:

$$h'_i = \big\|_{k=1}^K \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k)} W^{(k)} h_j \right) \quad (2.5)$$

where K represents the number of attention heads and $\|$ denotes concatenation. Multi-head attention improves representation learning and robustness.

Step 7: Output Layer and Prediction: The final node embeddings are passed to the output layer for tasks such as node classification, graph classification, link prediction, and regression.

Advantages of GATv2

GATv2 provides a dynamic attention mechanism that improves feature aggregation and representation learning compared to the original GAT architecture. It automatically identifies important neighboring nodes, performs effectively on irregular graph data, and improves robustness through multi-head attention, making it highly suitable for traffic prediction and autonomous navigation applications.

Limitations of GATv2

Despite its advantages, GATv2 requires high computational and memory resources due to attention computations on large graphs. Training time increases with graph size, and deep GATv2 architectures may suffer from over-smoothing and scalability issues for extremely large datasets.

GATv2 vs GAT

Graph Attention Network v2 (GATv2) was introduced as an improvement over the original Graph Attention Network (GAT) to overcome the limitation of static attention mechanisms. While both architectures use attention-based aggregation for graph representation learning, GATv2 provides a more expressive and adaptive attention mechanism.

In the original GAT architecture, the attention mechanism is defined as:

$$e_{ij} = \text{LeakyReLU} \left(a^T [W h_i \| W h_j] \right) \quad (2.6)$$

In GATv2, the order of operations is modified as:

$$e_{ij} = a^T \text{LeakyReLU}(W[h_i || h_j]) \quad (2.7)$$

This modification allows the attention scores to dynamically depend on both source and neighboring nodes, improving feature interaction learning and representation capability.

Table 2.1: Comparison Between GAT and GATv2

Feature	GAT	GATv2
Attention Type	Static Attention	Dynamic Attention
Neighbor Ranking	Fixed	Context-dependent
Expressiveness	Limited	More expressive
Flexibility	Lower	Higher
Performance	Good	Improved on many graph tasks

GATv2 is preferred over GAT because its dynamic attention mechanism enables better learning of complex graph relationships, resulting in improved feature aggregation, representation learning, and prediction accuracy in graph-based applications.

2.2.2 Long Short-Term Memory (LSTM)

Introduction to LSTM

Long Short-Term Memory (LSTM) is an advanced Recurrent Neural Network (RNN) architecture designed to learn long-term dependencies in sequential data. It was introduced to overcome the vanishing gradient problem faced by traditional RNNs through the use of memory cells and gating mechanisms. LSTM can selectively remember, update, and forget information, making it highly effective for applications such as traffic prediction, speech recognition, natural language processing, and time-series forecasting.

Fundamental Concept of LSTM

The primary objective of LSTM is to preserve important sequential information over long time intervals using memory cells and gating mechanisms. The architecture controls information flow through input, forget, and output gates, enabling efficient learning of temporal dependencies.

Architecture and Working of LSTM

The overall workflow of LSTM consists of gate computation, memory cell update, and hidden state generation.

Input Gate, Forget Gate, and Output Gate

At each time step, the current input and previous hidden state are used to compute the input, forget, and output gates using sigmoid activation functions. These gates regulate information flow into and out of the memory cell.

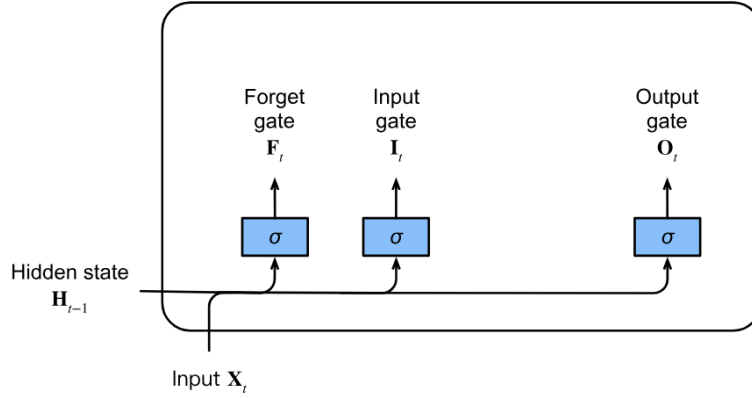


Figure 2.3: Computing the input gate, forget gate, and output gate in an LSTM model

The gates are mathematically represented as:

$$\begin{aligned} \mathbf{I}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xi} + \mathbf{H}_{t-1} \mathbf{W}_{hi} + \mathbf{b}_i) \\ \mathbf{F}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xf} + \mathbf{H}_{t-1} \mathbf{W}_{hf} + \mathbf{b}_f) \\ \mathbf{O}_t &= \sigma(\mathbf{X}_t \mathbf{W}_{xo} + \mathbf{H}_{t-1} \mathbf{W}_{ho} + \mathbf{b}_o) \end{aligned} \tag{2.8}$$

where $\mathbf{I}_t$, $\mathbf{F}_t$, and $\mathbf{O}_t$ represent the input, forget, and output gates respectively, while $\mathbf{X}_t$ and $\mathbf{H}_{t-1}$ denote the current input and previous hidden state.

Input Node

The candidate memory information is computed using the **tanh** activation function as:

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t \mathbf{W}_{xc} + \mathbf{H}_{t-1} \mathbf{W}_{hc} + \mathbf{b}_c) \tag{2.9}$$

where $\tilde{\mathbf{C}}_t$ represents the candidate memory state.

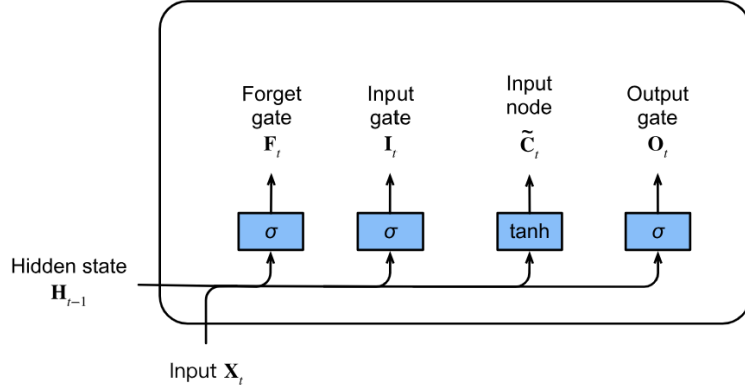


Figure 2.4: Computing the input node in an LSTM model

Memory Cell Internal State

The memory cell state is updated using the forget gate and input gate as:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t \quad (2.10)$$

where $\mathbf{C}_t$ represents the updated memory cell state and $\odot$ denotes element-wise multiplication. This mechanism helps preserve important long-term information while discarding irrelevant data.

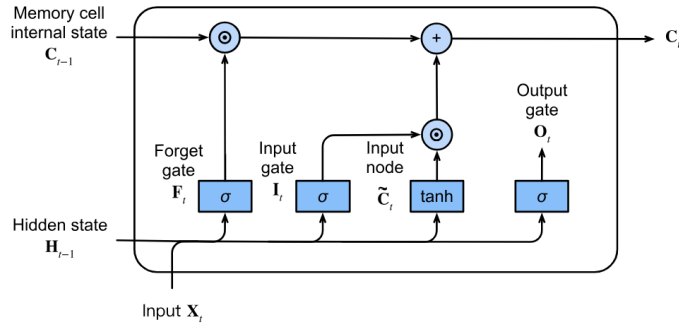


Figure 2.5: Computing the memory cell internal state in an LSTM model

Hidden State

The hidden state output is computed as:

$$\mathbf{H}_t = \mathbf{O}_t \odot \tanh(\mathbf{C}_t) \quad (2.11)$$

where $\mathbf{H}_t$ represents the hidden state at time step t . The output gate controls how much information from the memory cell is passed to the next layer.

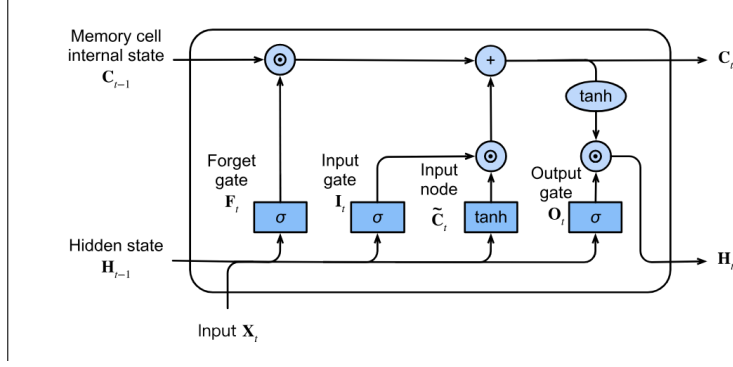


Figure 2.6: Computing the hidden state in an LSTM model

Advantages of LSTM

LSTM effectively solves the vanishing gradient problem and learns long-term sequential dependencies using memory cells and gating mechanisms. It performs efficiently on sequential and time-series data and is widely used in applications such as traffic prediction, speech recognition, machine translation, and forecasting.

Limitations of LSTM

Despite its advantages, LSTM is computationally expensive due to multiple gates and memory operations. Training requires higher computational resources and longer execution time, and performance may decrease for extremely long sequences or insufficient training data.

Source: Adapted from “Dive into Deep Learning, Long Short-Term Memory (LSTM).”

2.3 Federated Learning

Federated Learning (FL) is a decentralized machine learning approach where multiple devices or organizations collaboratively train a shared model without exchanging raw data. Each client performs local training on its own dataset and shares only model parameters with a central server, enabling privacy-preserving collaborative learning across distributed environments.

2.3.1 Introduction

Traditional machine learning systems rely on centralized training, where data from multiple users or organizations is collected and stored on a central server for model development. Although effective, this approach raises significant concerns related to data privacy, security, and regulatory compliance. Federated Learning addresses these chal-

lenges by enabling collaborative model training without sharing confidential or sensitive data.

In Trade-Based Money Laundering (TBML) detection systems, organizations such as banks and financial institutions can locally train models using transaction records and suspicious trade patterns while sharing only model parameters with a central server. Introduced by Google in 2016 for Gboard applications, Federated Learning has emerged as an important privacy-preserving approach for distributed machine learning systems .

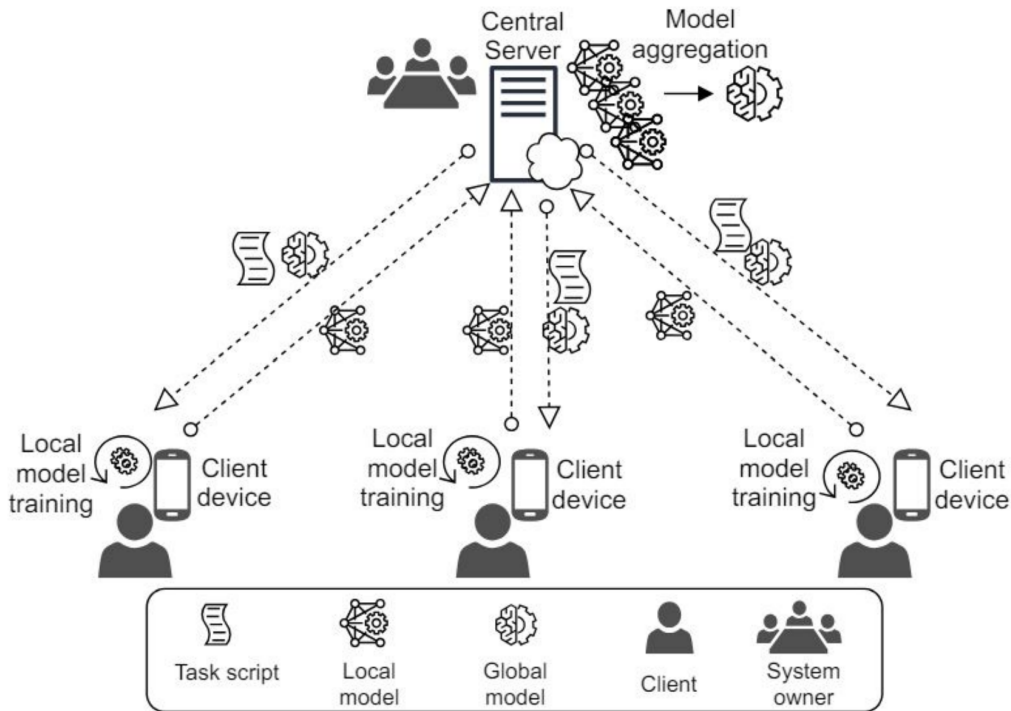


Figure 2.7: General Outlook of Federated Learning

Federated Learning also reduces communication overhead because only compact model updates are exchanged instead of complete datasets. This makes FL suitable for large-scale distributed environments involving mobile devices, IoT systems, edge devices, and financial institutions [14].

2.3.2 Working of Federated Learning

Federated Learning operates through iterative client-server interactions known as federated rounds. During each round, the central server distributes the current global model to selected clients, local training is performed independently on local datasets, and the updated model parameters are aggregated to generate an improved global model.

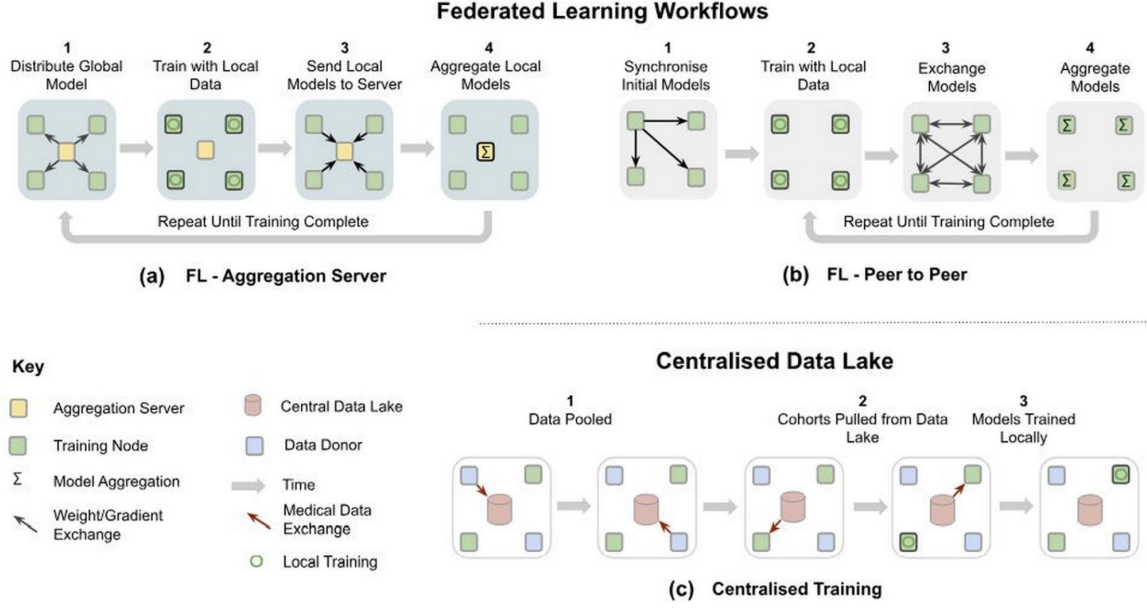


Figure 2.8: Federated Learning Workflows

Step 1: Initialization An initial machine learning model is selected and initialized at the central server. Client devices or institutional servers are then prepared to participate in local training.

Step 2: Client Selection During each federated round, only a subset of clients is selected for training. These clients receive the latest global model parameters from the server.

Step 3: Local Training Each selected client trains the received model locally using its private dataset without sharing raw transactional or customer information.

Step 4: Reporting and Aggregation After local training, clients send only model parameters or gradients to the aggregation server. The server combines these updates using aggregation algorithms such as Federated Averaging (FedAvg).

The Federated Averaging process is represented as:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_k^{(t+1)} \quad (2.12)$$

where w_{t+1} represents the updated global model, $w_k^{(t+1)}$ denotes the local model of client k , n_k represents the number of training samples at client k , and n denotes the total

number of training samples across all clients.

Step 5: Termination The federated training process continues iteratively until the desired accuracy, convergence criteria, or maximum communication rounds are achieved.

2.3.3 Advantages of Federated Learning

Federated Learning improves data privacy and security by ensuring that raw data remains within local devices or organizations. It supports regulatory compliance, enables collaborative intelligence without direct data sharing, reduces communication overhead, and improves model generalization using diverse distributed datasets. FL is also scalable and supports real-time learning on edge devices and organizational servers.

2.3.4 Limitations of Federated Learning

Despite its advantages, Federated Learning introduces challenges such as communication delays, high computational requirements, non-i.i.d. data distributions, client failures, synchronization issues, and security threats such as model poisoning and backdoor attacks. Variations in client hardware capabilities and unstable network connections can also affect training efficiency and global model convergence.

2.4 User Interface

Modern financial monitoring platforms increasingly rely on scalable web application technologies to support real-time transaction analysis, fraud monitoring, and interactive visualization of financial activities. The rapid growth of digital banking and online financial services has significantly increased the demand for responsive, secure, and data-driven monitoring systems capable of handling large volumes of transactional information efficiently. Consequently, modern Anti-Money Laundering systems employ component-based web architectures and client-server communication models to provide efficient user interaction and real-time analytical capabilities.

Advanced frontend frameworks and backend service architectures play a critical role in enabling modular application development, dynamic rendering, secure authentication, and seamless integration with fraud detection engines. Such technologies support the visualization of suspicious transactional activities, alert generation, role-based access control, and automated compliance workflows within financial institutions. The integration of scalable user interface technologies with intelligent analytical systems further enhances

operational efficiency and supports real-time financial crime investigation processes.

2.4.1 Modern Web Application Frameworks

Modern web application frameworks provide the architectural foundation required for developing scalable, responsive, and interactive software systems. Traditional web applications primarily relied on static page rendering and tightly coupled server-side architectures, which limited dynamic user interaction and real-time data processing capabilities. In contrast, modern frameworks adopt component-based and asynchronous rendering approaches that improve application scalability, maintainability, and user experience within large-scale distributed systems.

Frontend frameworks such as React.js enable the development of reusable and modular user interface components capable of dynamically updating application states without requiring complete page reloads. Such frameworks employ virtual Document Object Model (DOM) mechanisms and state-driven rendering principles to efficiently manage user interactions and real-time data visualization. This architectural approach significantly improves responsiveness and enables scalable dashboard development for data-intensive applications such as financial monitoring systems.

Server-side rendering and hybrid rendering frameworks such as Next.js further enhance modern web application performance by combining client-side interactivity with optimized server-side content delivery. These frameworks support efficient routing, dynamic data fetching, and improved application scalability while reducing rendering overhead for large-scale systems. Consequently, modern web application frameworks have become an essential foundation for building secure, scalable, and real-time financial monitoring platforms.

2.4.2 Component Lifecycle and Reactivity

Modern frontend frameworks like React and Next.js use reactive rendering methods to synchronize user interfaces with changes in application state and backend responses. This methodology enables effective visualization of transaction analytics, fraud alerts, compliance activities, and real-time monitoring information in the proposed TBML detection framework, while providing a responsive and scalable environment for dashboards. The dashboard is server-side pre-rendered to deliver the initial content to the user, improving loading performance and reducing processing overhead on the client.

When the interface is loaded, React hydration kicks in, allowing you to add interactive functionalities like event handling, communication with APIs, and state management. With changes in transactional data and analytics output, React re-renders only the components that are affected using Virtual DOM reconciliation, ensuring efficient rendering and real-time synchronization.

2.5 Summary

This chapter presented the theoretical foundations needed for Trade-Based Money Laundering (TBML) detection, including laundering methods, the limits of conventional Anti-Money Laundering systems, graph-based financial modeling, heterogeneous Graph Neural Networks, Graph Attention mechanisms, and federated learning. It also discussed real-time financial monitoring architectures and visualization systems used for fraud analytics and compliance operations. Taken together, these ideas form the analytical and computational base for the proposed privacy-preserving TBML detection framework developed in the later chapters.

Chapter 3

Design of the Proposed TBML Detection Framework

CHAPTER 3

DESIGN OF THE PROPOSED TBML DETECTION FRAMEWORK

This chapter presents the design of the proposed TBML detection framework. It brings together the software requirements, overall architecture, and module-level design, then walks through the operational requirements and user roles, the system workflows, graph-construction approach, federated learning setup, data-flow representations, and the analytical modules used for detection, investigation, and reporting.

3.1 Software Requirements Specifications

The software requirements for the proposed TBML detection framework describe what the system needs in order to operate reliably in a real institutional setting. They cover the functional behavior of the platform, the quality attributes it must maintain, the software stack it depends on, and the hardware resources needed to support fraud analysis, transaction monitoring, secure access, and compliance management. The details are grouped into functional requirements, non-functional requirements, software requirements, and hardware requirements in the following subsections.

3.1.1 Overall Description

The proposed framework is built as a privacy-preserving financial monitoring platform for identifying suspicious Trade-Based Money Laundering activity across large-scale transactional ecosystems. It combines heterogeneous graph-learning mechanisms, federated analytical workflows, role-based monitoring dashboards, and automated Suspicious Transaction Report (STR) generation within a centralized compliance-monitoring environment. In practice, the framework supports real-time transaction monitoring, graph-based fraud classification, institutional investigation workflows, and regulator-side analytical visualization for exposing hidden laundering relationships across connected financial networks.

Product Perspective

The proposed TBML detection framework acts as a financial monitoring and fraud-analysis platform that brings heterogeneous graph modeling, Graph Neural Network-based analytics, federated learning, and compliance monitoring into one environment. It

represents financial entities such as accounts, organizations, invoices, and transactions as connected graph structures so that hidden relationships and suspicious laundering patterns can be captured more effectively than with conventional rule-based Anti-Money Laundering systems. Role-based dashboards are also included to support transaction monitoring, fraud review, alert management, and automated Suspicious Transaction Report (STR) generation.

Product Functions

The primary functionalities supported by the proposed framework are summarized as follows:

- **Transaction Monitoring:** The system continuously monitors financial transactions and trade-related activities to identify suspicious transactional behavior within large-scale financial ecosystems.
- **Graph-Based Financial Analysis:** Financial entities such as accounts, organizations, invoices, and transactions are represented as interconnected graph structures for relational analysis and hidden dependency detection.
- **Fraud Classification and Risk Prediction:** Graph Neural Network-based analytical models classify suspicious transactions and generate fraud risk predictions based on transactional relationships and behavioral patterns.
- **Federated Learning Support:** The framework supports collaborative model training across multiple institutional environments while preserving sensitive financial data privacy.
- **Real-Time Dashboard Visualization:** Interactive monitoring dashboards provide visualization of transaction analytics, fraud alerts, risk distributions, and suspicious activity summaries for compliance analysts.
- **Suspicious Transaction Report (STR) Generation:** The system automatically generates structured Suspicious Transaction Reports for identified fraudulent transactions to support compliance and regulatory workflows.
- **Authentication and Role-Based Access Control:** Secure authentication and controlled access mechanisms are incorporated to manage user privileges and protect

sensitive financial information within the monitoring platform.

User Characteristics

The proposed TBML detection framework supports multiple categories of institutional users involved in fraud monitoring, compliance verification, and regulatory supervision activities. Since the system processes sensitive transactional and compliance-related information, role-based access mechanisms are enforced to ensure secure and controlled interaction with the platform. The major user categories associated with the proposed framework are summarized as follows:

- **Bank Compliance Team:** The bank compliance team represents institutional analysts responsible for monitoring transactional activities and investigating suspicious financial behavior identified by the fraud-detection framework. These users are expected to possess domain knowledge related to Anti-Money Laundering (AML) procedures, transaction analysis, and institutional compliance operations. Authorized compliance personnel interact with analytical dashboards, review suspicious transaction records, verify fraud alerts, and manage institutional STR escalation workflows.
- **Regulatory Administrator:** The regulatory administrator functions as a supervisory authority responsible for overseeing fraud-monitoring activities across participating financial institutions. These users typically possess advanced regulatory and compliance expertise associated with financial investigation and institutional auditing operations. Regulatory administrators access consolidated Suspicious Transaction Reports (STRs), monitor institution-level compliance activities, and supervise large-scale suspicious transactional behavior across distributed banking environments.

Constraints

The development and deployment of the proposed TBML detection framework are subject to multiple operational, computational, and regulatory constraints associated with financial monitoring systems. These limitations influence the scalability, analytical performance, and real-time operational capabilities of the framework.

The major constraints associated with the proposed system are summarized as follows:

-
- **Data Privacy and Regulatory Constraints:** Financial institutions operate under strict privacy regulations and compliance policies that restrict direct sharing of sensitive transactional information across organizations.
 - **Large-Scale Transactional Data Processing:** The system must process large volumes of interconnected financial transactions and graph relationships, which may increase computational complexity and memory utilization during real-time analysis.
 - **Availability of Real-World TBML Datasets:** Access to publicly available Trade-Based Money Laundering datasets is limited due to confidentiality and regulatory restrictions, making large-scale real-world experimentation challenging.
 - **Dynamic Fraud Patterns:** Money laundering strategies continuously evolve over time, requiring fraud detection models to adapt to changing transactional behaviors and emerging laundering techniques.
 - **Infrastructure and Computational Limitations:** Training graph-based machine learning models and processing heterogeneous financial networks require significant computational resources, especially for large-scale transactional ecosystems.

3.1.2 Specific Requirements

The specific requirements define the operational behavior, performance expectations, software dependencies, and computational resources needed for the framework to run efficiently. They make sure the system can support real-time fraud analysis, scalable monitoring, secure interaction, and intelligent compliance management within financial institutions.

Functional Requirements

The functional requirements define the major operations and services that must be supported by the proposed TBML detection framework. The primary functional requirements of the system are summarized as follows:

- The system shall enforce role-based authorization policies to regulate operational access privileges for compliance analysts and regulatory administrators.

-
- The framework shall support ingestion and preprocessing of transactional datasets for heterogeneous financial graph construction.
 - The proposed system shall model financial entities, transactions, and trade relationships as interconnected heterogeneous graph structures.
 - The analytical engine shall perform graph-based fraud classification and generate transaction-level risk predictions using federated learning methodologies.
 - The dashboard infrastructure shall support real-time visualization of suspicious transactions, fraud alerts, analytical summaries, and compliance-monitoring activities.
 - The system shall enable institutional compliance teams to review suspicious transactions and perform fraud escalation or clearance procedures.
 - The framework shall provide features to generate Suspicious Transaction Reports (STRs) for transactions identified as potentially fraudulent.
 - The regulator-side monitoring interface shall support centralized supervision of institutional fraud reports and suspicious financial activities.
 - The analytical interface shall provide transaction-filtering, search, and investigative querying functionalities for fraud-analysis workflows.

Non-Functional Requirements

The non-functional requirements define the quality attributes, operational constraints, and performance expectations associated with the proposed TBML detection framework. These requirements ensure that the system remains scalable, secure, reliable, and responsive while processing large-scale financial transactions and analytical workloads.

The primary non-functional requirements associated with the proposed system are summarized as follows:

- **Scalability:** The framework should support scalable processing of large transactional datasets and heterogeneous graph structures without significant degradation in analytical performance.

-
- **Real-Time Processing:** The system should support low-latency transaction analysis and fraud prediction to enable continuous real-time monitoring of suspicious financial activities.
 - **Security and Privacy:** The framework should ensure secure access control, protected communication, and privacy-preserving handling of sensitive financial information and compliance-related data.
 - **Reliability:** The system should maintain stable analytical performance and continuous monitoring capabilities during prolonged operational workloads and concurrent user access.
 - **Availability:** The monitoring platform should provide consistent access to dashboards, fraud analytics, and reporting services for authorized institutional users.
 - **Maintainability:** The system architecture should support modular development, efficient debugging, and simplified integration of additional analytical modules and monitoring functionalities.
 - **Usability:** The dashboard interfaces and visualization systems should provide intuitive interaction mechanisms for compliance analysts and regulatory administrators during fraud investigation activities.

Software Requirements

The implementation of the proposed TBML detection framework depends on a combination of software technologies, machine learning libraries, backend services, database systems, and visualization tools. Together, these components support graph-based fraud detection, federated learning, real-time transaction monitoring, and compliance management. The framework therefore needs coordinated interaction between the frontend dashboard, backend processing services, database layers, and machine learning environments. The software requirements are summarized in Table ??.

Table 3.1: Software Requirements of the Proposed TBML Detection Framework

Category	Technologies / Tools	Purpose in the System
Operating System	Windows 11	Provides the development and deployment environment for backend services, databases, and analytical modules.
Programming Languages	Python, JavaScript	Used for backend development, machine learning workflows, API services, and frontend application development.
Frontend Frameworks	Next.js, React.js, Tailwind CSS	Supports responsive user interface development, dashboard visualization, and real-time monitoring functionalities.
Backend Framework	FastAPI	Facilitates asynchronous API communication between frontend dashboards and backend fraud detection services.
Database Systems	PostgreSQL, Memgraph	Stores transactional records, graph relationships, compliance information, and analytical data required for fraud analysis.
Machine Learning Frameworks	PyTorch, PyTorch Geometric	Supports deep learning operations, Graph Neural Network implementation, and graph-driven fraud analysis.
Federated Learning Framework	Flower	Enables distributed federated learning and privacy-preserving model aggregation across institutional environments.
Streaming Platform	Apache Kafka	Supports real-time transaction streaming and event-driven processing within the monitoring pipeline.
Containerization Platform	Docker	Provides containerized deployment and scalable execution environments for system integration.
LLM Integration Service	Groq LLM API	Supports automated generation of Suspicious Transaction Reports and compliance-oriented analytical summaries.

Hardware Requirements

The implementation of the proposed TBML detection framework also needs a defined hardware setup to support graph processing, machine learning execution, federated coordination, real-time transaction monitoring, and dashboard visualization. The required hardware is summarized in Table ??.

Table 3.2: Hardware Requirements of the Proposed TBML Detection Framework

Hardware Component	Minimum Requirement	Purpose in the System
Processor	Intel Core i7 / AMD Ryzen 7	Supports efficient execution of graph analytical models, backend services, and machine learning computations.
RAM	16 GB	Supports graph datasets, real-time analytical workloads, and concurrent monitoring operations.
GPU	NVIDIA RTX 3050	Accelerates Graph Neural Network training and deep learning computations.
Storage	512 GB SSD	Stores transactional datasets, graph databases, analytical models, and generated compliance reports.

3.2 High level Design

This section outlines the high-level design of the proposed TBML detection framework. It covers the main architectural choices, the design considerations behind them, and the overall system layout that brings graph-based fraud detection, federated learning, and real-time monitoring together in a single compliance platform.

3.2.1 Design Considerations

The design of the proposed TBML detection framework is shaped by operational, analytical, and security requirements that are common in large-scale financial monitoring systems. Because Trade-Based Money Laundering involves connected financial entities, cross-border transactions, and hidden transactional dependencies, the architecture has to support scalable analytical processing, secure institutional collaboration, and efficient graph-based fraud analysis. The major design considerations are discussed in the following subsections.

General Considerations

The proposed TBML detection framework is designed to support large-scale transaction monitoring and fraud analysis across distributed financial environments. Since laundering activities typically involve interconnected accounts, trade entities, invoices, and transactional relationships, the framework uses graph-based modeling to represent hidden financial dependencies more effectively than conventional relational systems. The architecture also supports real-time transaction processing and scalable analytical workflows as institutional transaction volumes continue to grow.

Security, privacy preservation, and scalability are central concerns in the proposed architecture because of the sensitive nature of financial and compliance data. To address them, the framework uses secure authentication, role-based access control, and federated learning so institutions can collaborate without sharing raw transactional data. The system is also built with a modular structure to make integration, maintenance, and future extension of monitoring, fraud detection, compliance reporting, and visualization services easier.

Development Methods

The proposed TBML detection framework is developed using a modular and iterative approach so that the system can be integrated cleanly and its analytical functions refined step by step. Individual components such as transaction monitoring, graph construction, fraud detection, federated learning, and dashboard interfaces are built and tested separately before being brought together in the full monitoring framework.

The development process also includes incremental model evaluation and experimentation to improve fraud detection performance and analytical reliability. Graph-based machine learning models are trained on processed transactional datasets, while frontend and backend services are connected through API-based communication to support real-time interaction between monitoring modules and analytical services.

Version control and collaborative development practices are used throughout the implementation process to manage backend services, machine learning workflows, frontend interfaces, and analytical settings efficiently. This modular approach also makes debugging, testing, maintenance, and future enhancement much more manageable.

3.2.2 Architectural Strategies

The architectural strategies adopted in the proposed TBML detection framework are aimed at supporting scalable financial transaction analysis, secure institutional collaboration, and smooth interaction between analytical services and monitoring interfaces. The framework brings graph-based fraud detection models, backend analytical services, real-time dashboards, and distributed learning mechanisms together in a unified monitoring environment. The main architectural strategies are discussed in the following subsections.

Programming Language

The proposed TBML detection framework primarily uses Python for backend services, graph processing, machine learning workflows, and analytical computation. Python was chosen because it has strong support for deep learning libraries, graph analytical frameworks, and federated learning environments. Libraries such as PyTorch, PyTorch Geometric, and Flower are used to implement the graph neural network models and distributed learning mechanisms within the fraud detection framework.

The frontend monitoring dashboard is developed using Next.js and React.js to support responsive user interface design and real-time analytical visualization. JavaScript and TypeScript are used to implement dashboard components, transaction monitoring interfaces, and compliance management services. FastAPI is also integrated to support API-based communication between frontend dashboards, backend services, and machine learning modules.

System Integration Architecture

The proposed framework follows a modular client-server and service-oriented integration architecture so that the analytical modules, backend services, and frontend monitoring interfaces can work together in a coordinated way. The fraud detection models, transaction monitoring services, authentication modules, and compliance reporting components operate as independent services while staying synchronized through backend APIs and shared database environments.

The machine learning modules responsible for graph processing and fraud prediction run separately from the frontend monitoring dashboard. Processed fraud predictions, transaction classifications, and generated Suspicious Transaction Reports are synchronized through backend services so that the dashboard can display them in real time and support compliance management. This modular integration approach makes the frame-

work easier to scale, maintain, and extend.

Data Storage Management

The proposed TBML detection framework uses both relational and graph-based database systems to manage financial data efficiently and support analytical processing. Structured transactional records, authentication information, compliance-related data, and generated Suspicious Transaction Reports are stored in PostgreSQL. This relational database layer provides secure storage and efficient retrieval of operational monitoring data for the framework.

For the graph database environment, Memgraph is used to store the connected financial entities and transactional relationships. Graph nodes and edges represent accounts, organizations, invoices, and transactions for relational fraud analysis and graph neural network processing. Using relational and graph storage together improves scalability, analytical flexibility, and synchronization between backend analytical services and monitoring dashboards.

3.2.3 Overall System Architecture

The proposed TBML detection framework combines transaction processing, heterogeneous graph construction, federated learning, and graph-based fraud analytics within a single monitoring environment. Transaction records, trade documents, KYC information, and institutional logs are processed and transformed into heterogeneous financial graphs that capture relationships among accounts, organizations, invoices, and transactions. These graph representations are then analyzed using a hybrid HGNN–LSTM architecture to learn structural dependencies and temporal transaction behavior associated with laundering activity.

To support privacy-preserving collaboration, participating institutions train locally and share only model updates through the federated learning environment. The aggregated global model is then used for transaction risk scoring, fraud classification, and compliance monitoring. High-risk transactions are forwarded to compliance-management services for alert generation, Suspicious Transaction Report (STR) creation, and regulator-side monitoring. The overall architecture of the proposed framework is illustrated in Figure ??.

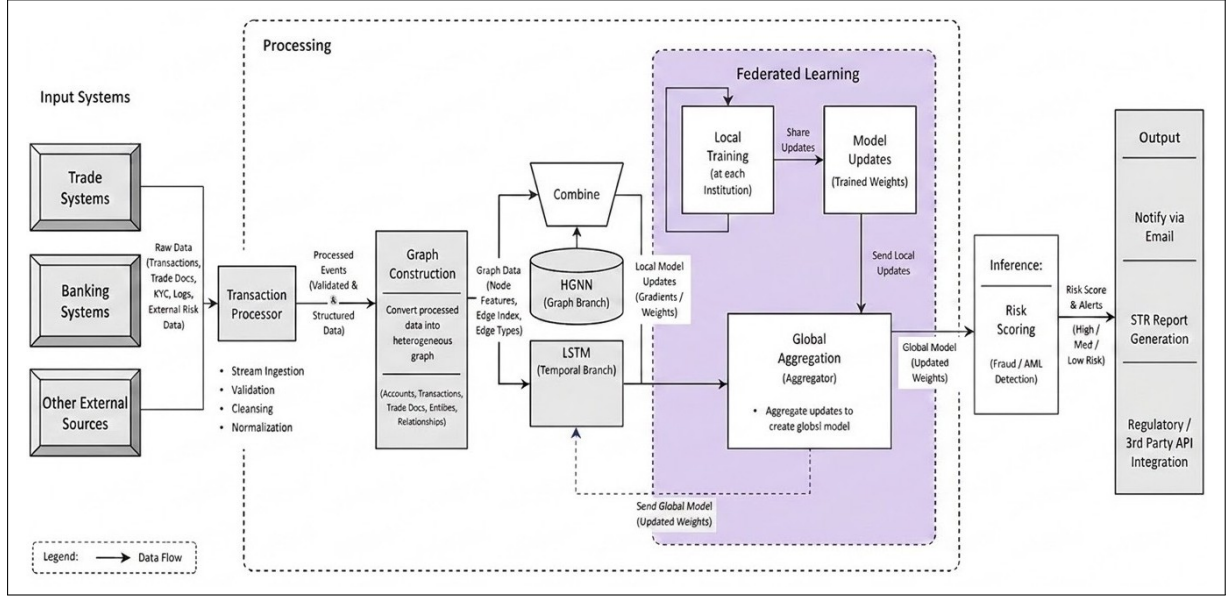


Figure 3.1: Overall Architecture of the Proposed TBML Detection Framework

3.2.4 Data Flow Diagrams

A Data Flow Diagram (DFD) is a simple way to show how data moves through a system and how processes, external entities, and data repositories relate to one another. DFDs make it easier to see how information enters the system, gets processed, is stored, and then moves to other components. In the proposed Federated Heterogeneous Graph Neural Network (HGNN)-Based Trade-Based Money Laundering (TBML) Detection System, they are used to represent transaction acquisition, graph construction, federated learning, fraud analysis, and alert generation. The split into Level 0, Level 1, and Level 2 diagrams gives a clearer view of the operational flow, the subsystem interactions, and the data-processing steps involved in real-time AML analytics. The diagrams follow the Yourdon and De Marco DFD methodology for consistency.

Data Flow Diagram Level 0

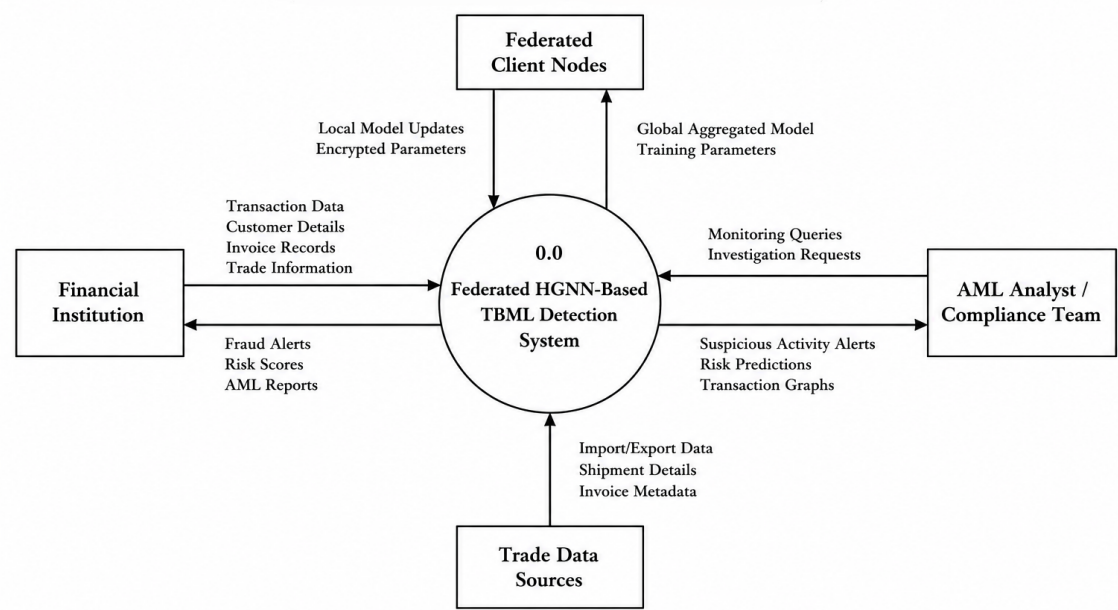


Figure 3.2: Level 0 Data Flow Diagram

The Level 0 Data Flow Diagram (DFD) of the designed Federated HGNN-Based TBML Detection System is shown in Figure ???. It gives a top-level view of the system as a single process interacting with external entities such as Financial Institutions, Federated Client Nodes, AML Analysts, and Trade Data Sources. The system receives transaction records, customer information, invoice details, and trade details from financial institutions, while trade data sources contribute import/export records and shipment data that help validate transactions and support graph construction. Federated client nodes exchange encrypted local model parameters with the global model, and the system produces fraud warnings, risk assessments, AML reports, and suspicious activity notifications for analysts and compliance teams.

Data Flow Diagram Level 1

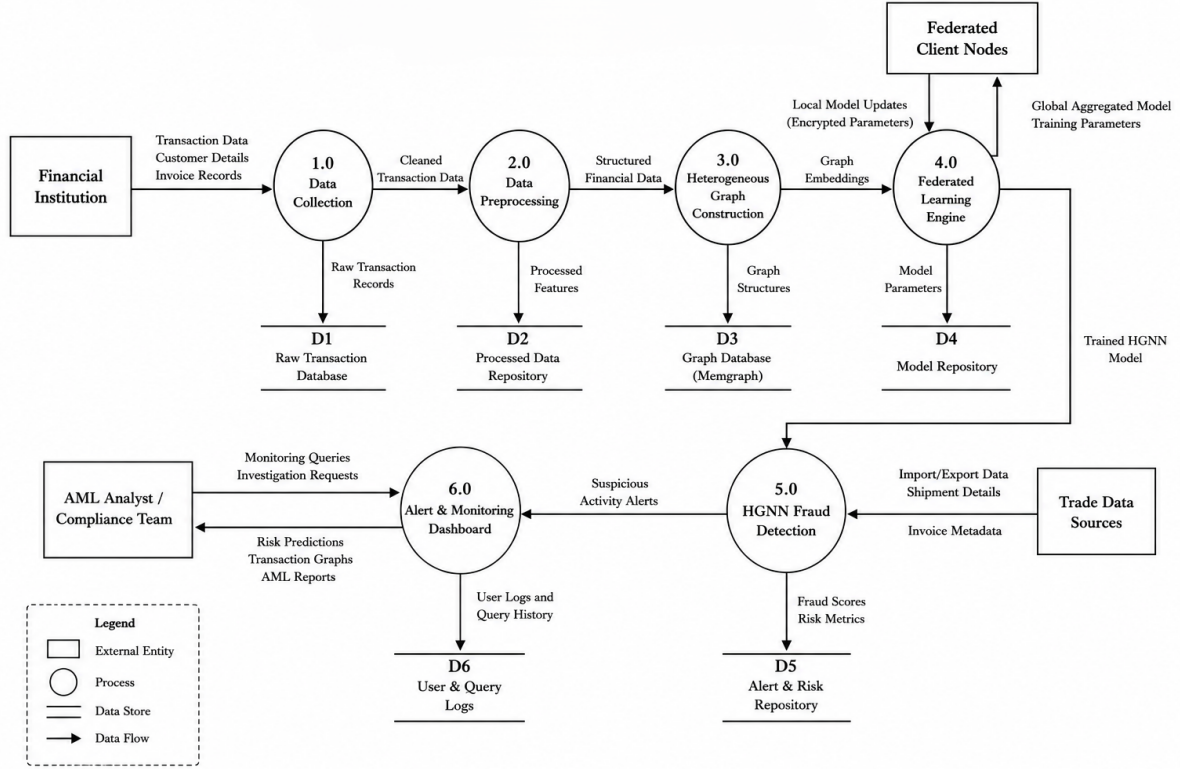


Figure 3.3: Level 1 Data Flow Diagram

The Level 1 Data Flow Diagram of the proposed TBML detection framework is shown in Figure ?? . It breaks the main system into functional processes such as Data Collection, Data Preprocessing, Heterogeneous Graph Construction, Federated Learning Engine, HGNN Fraud Detection, and the Alert and Monitoring Dashboard. The Data Collection module gathers raw transactional data and customer data from financial institutions. That data is cleaned and transformed during preprocessing to produce structured financial features. Those features are then used by the graph construction engine to build heterogeneous graph representations stored in the graph database. The federated learning engine trains distributed collaborative models using encrypted local updates from federated client nodes while preserving institutional data privacy. The trained HGNN model is used by the fraud detection engine to identify suspicious transactional patterns, compute fraud scores, and generate anomaly alerts. Finally, the monitoring dashboard allows AML analysts and compliance teams to investigate cases, visualize transaction graphs, and review risk assessment reports in real time.

Data Flow Diagram Level 2

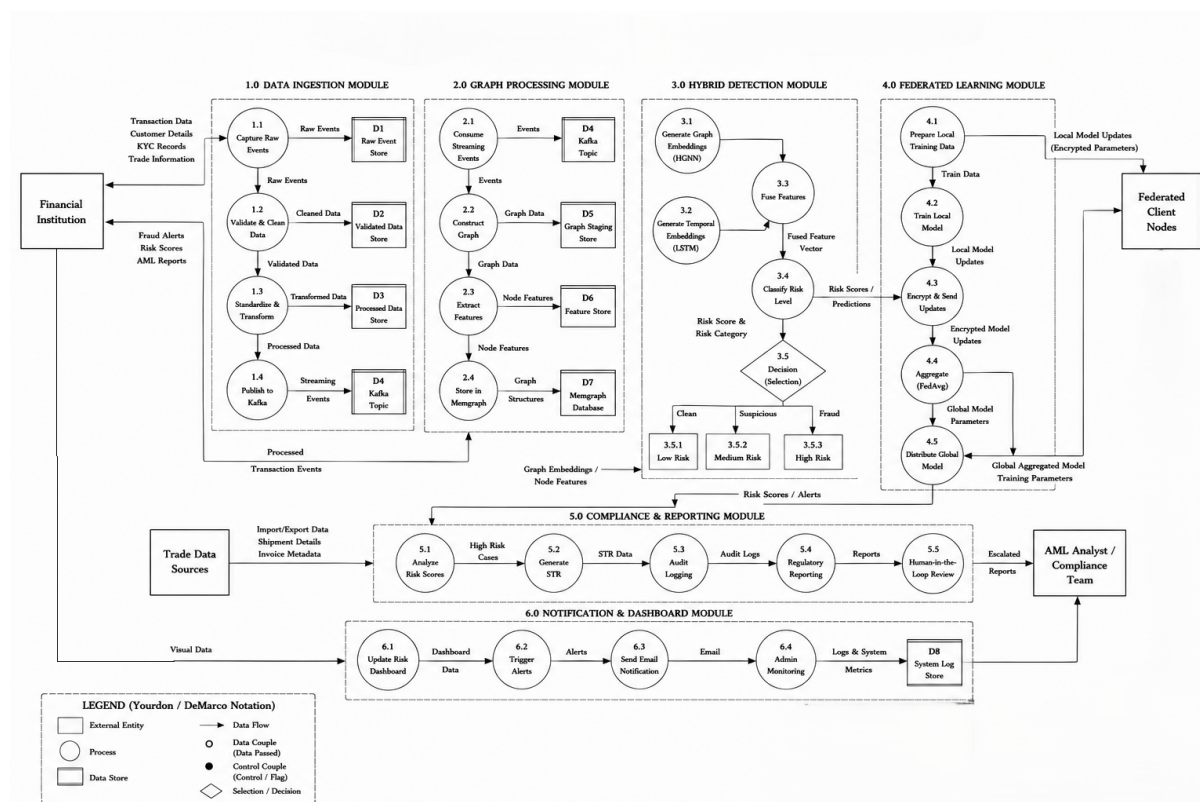


Figure 3.4: Level 2 Data Flow Diagram

Figure ?? shows the Level 2 DFD, which breaks down the HGNN Fraud Detection module in more detail. The process begins with Data Integration and Validation, where trade invoice data, shipment data, and transactional metadata are checked for inconsistencies. After validation, the data moves into Entity Resolution and Feature Extraction, which handles attribute extraction and prepares the features needed for graph-based learning. Heterogeneous Graph Generation then builds the financial transaction graphs that connect entities, account relations, invoice transactions, and other trade-related structures. HGNN Inference and Risk Scoring applies the graph neural network to generate scores and identify suspicious transactional behavior. The results then move through Anomaly Detection and Classification, followed by Alert Generation and Prioritization, which ranks suspicious transactions according to risk. The final stage, Result Aggregation and Reporting, produces fraud summaries, alert notifications, and reports that are sent to the Alert and Monitoring Dashboard for review.

3.3 Detailed System Design

This section gives a detailed view of the system design for the proposed TBML detection framework. It covers the structure chart showing how the system components are organized, the functional descriptions of the major modules, and the interaction workflow between the services and analytical components in the framework.

3.3.1 Structure Chart

The structure chart of the proposed TBML detection framework shows how the key functional modules are arranged. The framework includes frontend, backend, fraud detection, federated learning, database, and compliance layers, all of which support scalable monitoring and intelligent fraud analysis in Anti-Money Laundering ecosystems.

The frontend layer provides dashboard interfaces, transaction monitoring, risk visualization, and STR management. The backend layer coordinates transactions between the frontend interface and the analytical modules by handling APIs, transaction management, authentication, and authorization. The fraud detection layer contains the Hybrid HGNN-LSTM modules that handle graph analytics, transaction analysis, feature fusion, and risk prediction.

The federated learning layer supports collaborative privacy-preserving model training and global model synchronization among participants. The database layer manages transactions, graphs, audit logs, and compliance data. The compliance layer handles STR generation, regulatory reporting, email notifications, and monitoring processes. The structural organization of the proposed TBML detection framework is shown in Figure ??.

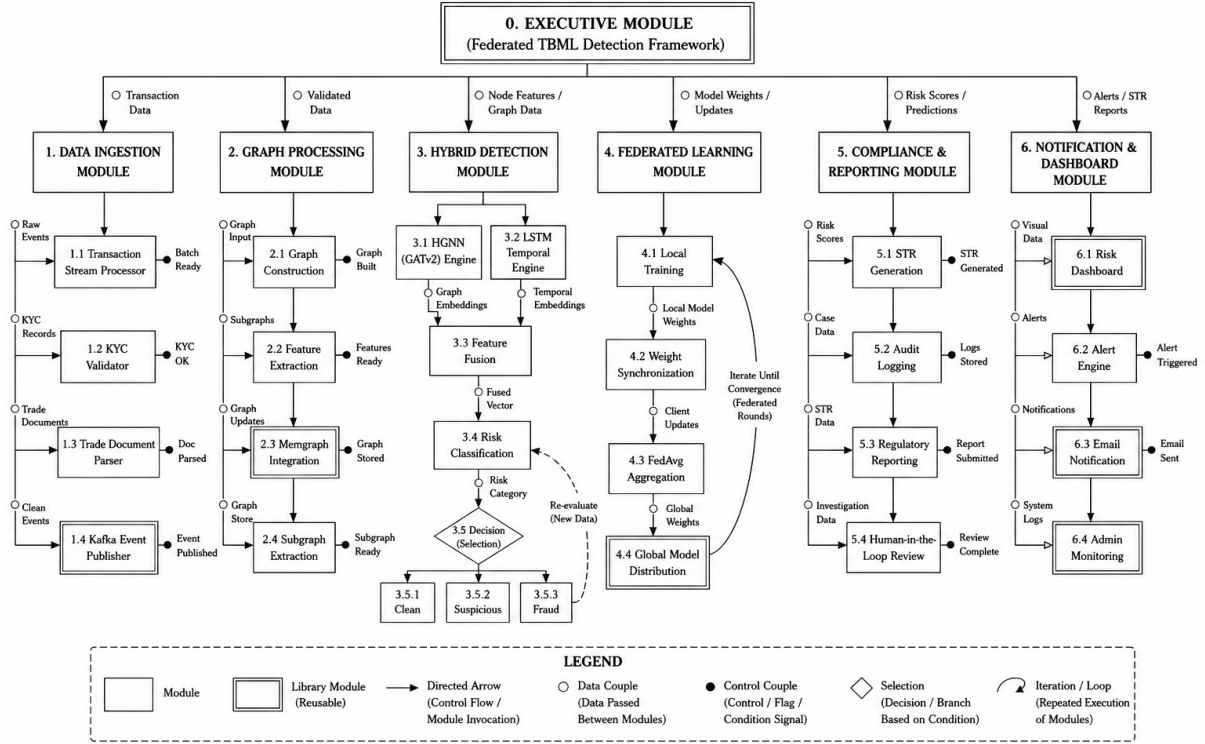


Figure 3.5: Structure Chart of the Proposed TBML Detection Framework

The structure chart makes the modular organization of the framework easier to understand, especially the interaction between the analytical, monitoring, storage, and compliance-oriented services. This modular design improves scalability, maintainability, service integration, and coordination between the frontend interfaces, backend analytical services, and distributed fraud detection environments.

3.3.2 Functional Description of the Modules

The functional description of the modules explains the internal processing workflow and interaction between the major analytical, monitoring, and compliance-oriented components operating within the proposed TBML detection framework. The framework integrates transaction streaming services, graph construction mechanisms, hybrid HGNN–LSTM analytical models, institutional verification environments, and regulatory reporting systems to support intelligent Trade-Based Money Laundering detection and real-time compliance monitoring.

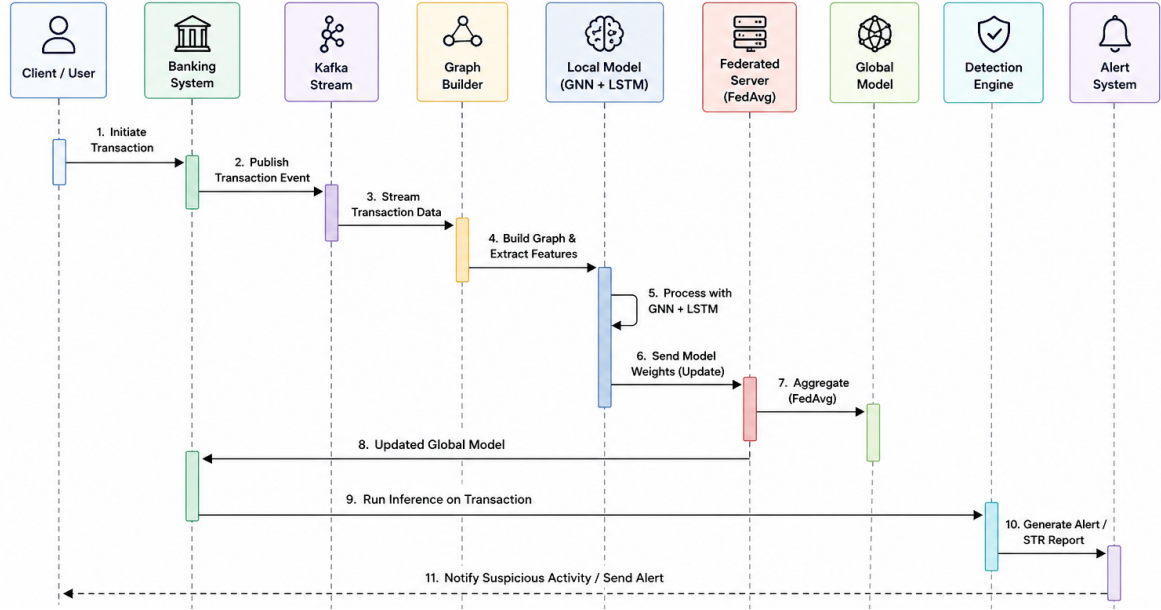


Figure 3.6: Workflow Interaction Diagram for the Proposed TBML Detection Framework

The workflow interaction diagram shows the coordinated interaction between streaming services, graph-based analytical modules, monitoring environments, institutional verification systems, and regulatory compliance services operating within the proposed framework. This modular workflow supports scalable transaction processing, intelligent fraud analysis, compliance management, and real-time Anti-Money Laundering monitoring across distributed financial environments.

Transaction Processing Module

The transaction processing Module acts as the initial ingestion environment within the proposed TBML detection framework. This Module collects financial transactions, KYC records, trade documents, and institutional monitoring events originating from banking systems and external financial environments.

The input data processed within this Module includes sender and receiver account identifiers, transaction amounts, commodity types, trade quantities, trade values, invoice details, transaction timestamps, and institutional monitoring attributes associated with financial activities. The collected transactional data is validated, normalized, and converted into structured transaction records to support reliable downstream analytical processing.

The processed transaction events are published into Kafka streaming topics to enable asynchronous communication between backend services and analytical modules. The

generated transaction streams serve as the primary input for graph construction, feature extraction, and graph-based fraud detection operations within the proposed framework.

Graph Construction Module

The graph construction module transforms processed transaction streams into heterogeneous financial transaction graphs for downstream graph-based analytical processing. The module receives validated transaction events from Kafka consumer services and models financial interactions between accounts, transactions, and trade documents within a connected graph environment.

The generated graph schema consists of interconnected node entities including `(:Account)`, `(:Transaction)`, and `(:Document)` connected through relationships such as `[ :SENDS ]`, `[ :TO ]`, and `[ :HAS_DOC ]` as illustrated in Figure ?? . The heterogeneous graph structure preserves transactional connectivity, intermediary account interactions, and trade-document relationships required for identifying hidden financial dependencies associated with Trade-Based Money Laundering activities.

The module interacts with the Memgraph database to store graph structures, entity relationships, and transactional connectivity information required for downstream feature extraction and fraud analysis. The generated graph representations and extracted relational features are subsequently forwarded to the Hybrid HGNN–LSTM fraud detection module for intelligent transaction classification.

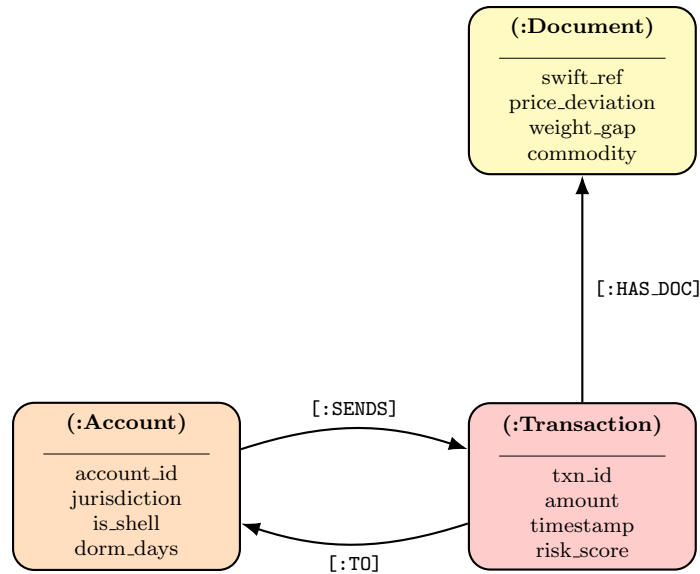


Figure 3.7: Heterogeneous graph schema representing entity attributes and transactional relationships in the proposed TBML detection framework.

Hybrid HGNN–LSTM Fraud Detection Module

The Hybrid HGNN–LSTM fraud detection module performs graph-based representation learning and temporal transaction analysis within the proposed TBML detection framework. The module receives KYC node features, graph edge relationships, SWIFT transaction attributes, sequential transaction tensors, and trade metadata extracted from the heterogeneous financial graph for downstream analytical processing.

The graph branch utilizes a three-layer GATv2 architecture to perform attention-based neighborhood aggregation and multi-hop message passing across interconnected financial entities. The stacked GATv2 layers improve two-hop neighborhood visibility and enable the model to capture hidden transactional relationships and intermediary account interactions associated with Trade-Based Money Laundering activities. The generated graph embeddings are globally pooled and fused with sequential transaction representations before being processed through the LSTM branch for temporal anomaly detection and transaction behavior analysis. In parallel, the trade branch processes trade-specific attributes including price deviation and weight gap scores using multilayer perceptron operations and feature normalization layers.

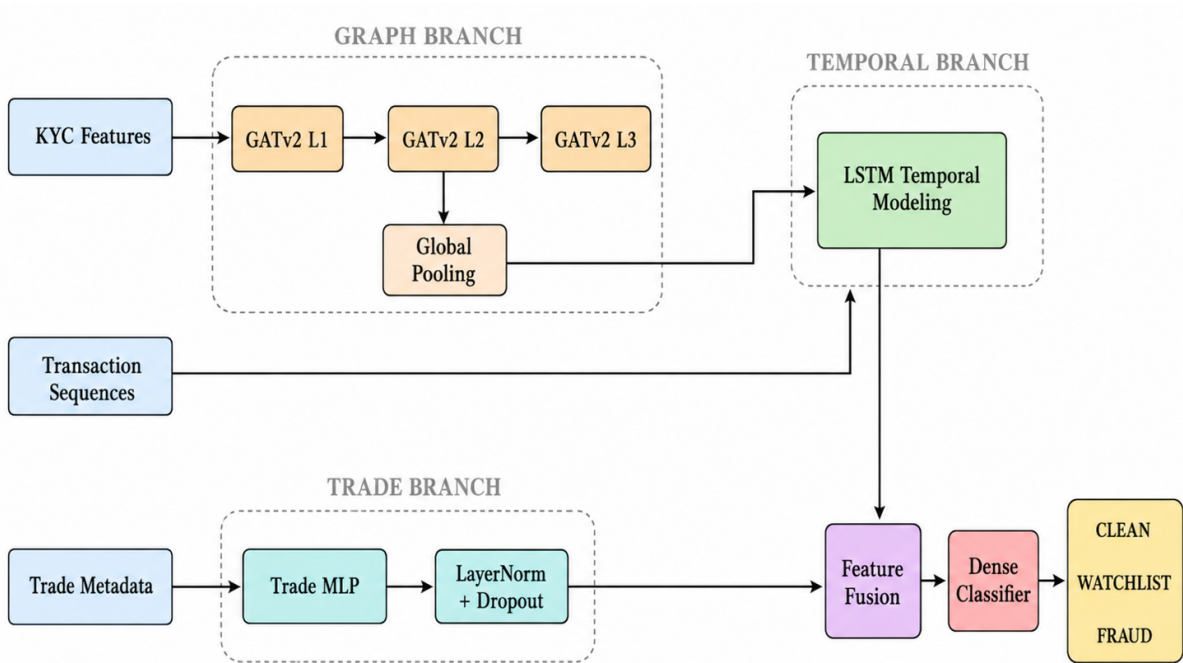


Figure 3.8: Architecture of the Proposed Hybrid HGNN–LSTM Fraud Detection Module

The generated graph embeddings, temporal representations, and trade feature vectors are combined within the feature fusion layer for downstream fraud prediction and

transaction risk analysis. The final dense classification layer categorizes transactions into predefined classes including **CLEAN**, **WATCHLIST**, and **FRAUD**. The overall architecture of the proposed Hybrid HGNN–LSTM fraud detection module is illustrated in Figure ??.

Federated Learning Module

The federated learning module enables privacy-preserving collaborative fraud detection across distributed financial institutions without directly sharing sensitive transactional data. Each participating institution performs local model training using institutional transaction graphs, transaction sequences, and fraud classification labels generated within the analytical environment. Independent federated clients maintain isolated training environments and perform localized optimization of the Hybrid HGNN–LSTM model on institution-specific financial datasets.

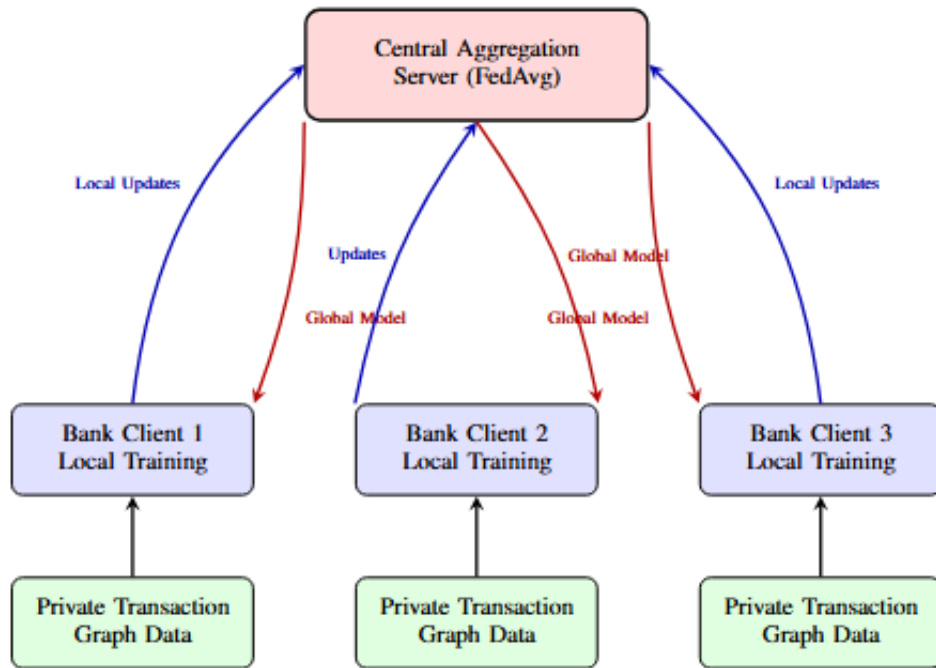


Figure 3.9: Federated learning architecture for privacy-preserving collaborative TBML detection across distributed institutions

The framework utilizes the Federated Averaging (**FedAvg**) strategy to aggregate distributed model parameters across participating institutions. During each federated communication round, locally trained model weights generated from individual institutional clients are transmitted to the central aggregation server where parameter averaging operations are performed to generate a globally optimized fraud detection model. The

aggregated global model parameters are subsequently redistributed to participating institutions for synchronized fraud detection and continuous real-time analytical inference. The overall federated learning architecture of the proposed framework is illustrated in Figure ??.

STR Generation and Notification Module

The STR generation and notification module is responsible for compliance reporting and fraud alert management within the proposed TBML detection framework. Transactions verified as fraudulent by authorized banking administrators are forwarded to this module for Suspicious Transaction Report (STR) generation and regulatory processing. The module generates STR reports containing transaction details, fraud classifications, trade metadata, institutional information, and investigation summaries associated with suspicious financial activities.

The Groq API service is utilized for analytical summarization and automated assistance during compliance documentation generation. Generated STR reports are subsequently forwarded to regulatory monitoring authorities for investigation and enforcement procedures. The notification management environment coordinates fraud alerts and institutional communication workflows, while the Resend email service delivers notification emails, investigation alerts, and regulatory enforcement messages to banking institutions and regulatory authorities.

Dashboard and Regulatory Monitoring Module

The dashboard and regulatory monitoring module provides real-time transaction visualization, fraud monitoring, and institutional investigation functionalities within the proposed TBML detection framework. The monitoring dashboard displays transaction classifications, fraud alerts, risk scores, STR records, and analytical monitoring information generated by the fraud detection environment. Separate dashboard environments are maintained for banking administrators and regulatory authorities to support institution-level monitoring and centralized compliance management operations.

For banking administrators, the dashboard provides a clear view of suspicious and fraudulent transactions flagged by the analytical model. They can review each case, verify the context, and then classify it as **CLEAN**, **WATCHLIST**, or **FRAUD**. This review step helps reduce false alarms while ensuring that genuine suspicious activity is escalated for further action. Transactions confirmed as fraudulent are then passed on for Suspicious

Transaction Report (STR) generation and regulatory reporting.

Regulatory authorities use a separate monitoring view to track generated STR reports and institution-level fraud cases in one place. From this interface, they can follow compliance activity, request supporting documents, initiate inquiry procedures, and carry out account freeze operations when necessary. The overall workflow is illustrated in Figure ??.

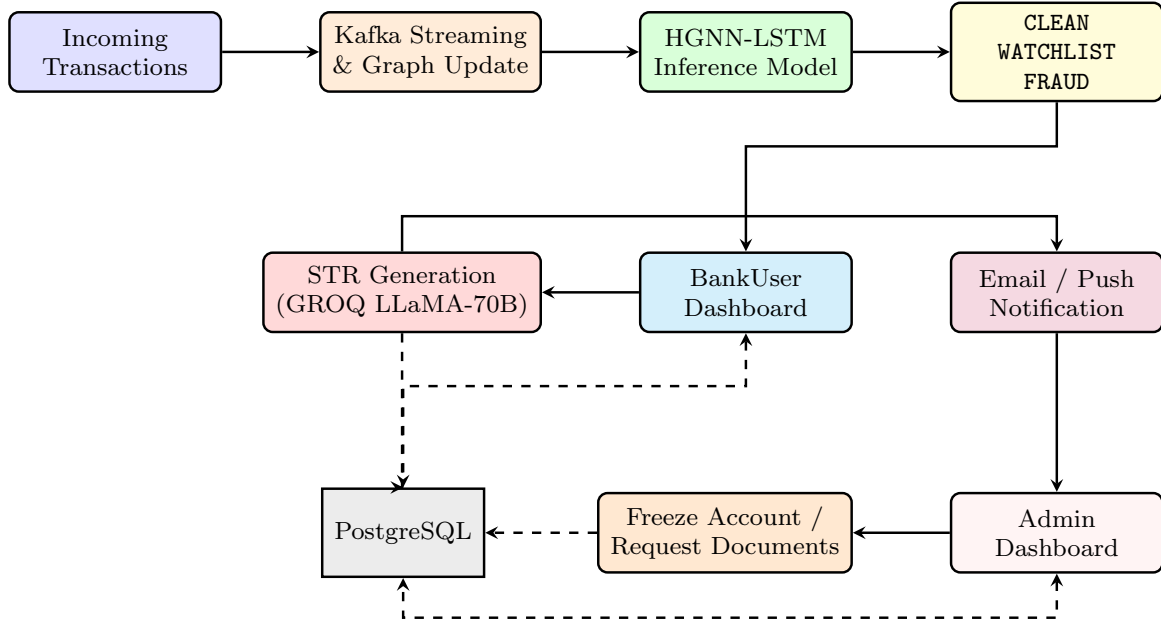


Figure 3.10: Real-time fraud inference and compliance workflow of the proposed TBML detection framework.

3.4 Summary

This chapter provided a detailed overview of the software specifications, architectural design, data flow diagrams, structure chart, and functional descriptions of the major modules in the proposed TBML detection framework. The discussion of the system architecture, module interactions, and analytical workflows gives a clearer picture of the operational mechanisms, service integration strategy, and real-time monitoring capabilities built into the framework for intelligent Trade-Based Money Laundering detection and compliance management across distributed financial environments.

Chapter 4

Implementation and Testing of the Proposed TBML Detection Framework

CHAPTER 4

IMPLEMENTATION AND TESTING OF THE PROPOSED TBML DETECTION FRAMEWORK

This chapter details the implementation of the FedGAT-TBML framework. It covers real-time transaction ingestion and streaming, construction and storage of heterogeneous transaction graphs, a hybrid HGNN–LSTM detector for sequence-aware graph inference, federated learning orchestration for privacy-preserving model updates, and a web dashboard for compliance monitoring and STR generation. The implementation leverages Apache Kafka for durable streaming, Memgraph for graph storage and queries, PostgreSQL for audit and STR records, PyTorch Geometric for graph neural network components (e.g., GATv2Conv), Flower for federated orchestration, FastAPI for backend services, and Next.js/TypeScript for the frontend.

4.1 Implementation

This section provides a comprehensive overview of the implementation of the proposed TBML detection framework, detailing the programming languages, frameworks, platforms, code conventions, system architecture, and the strategies employed to address implementation challenges. The implementation integrates real-time transaction streaming, heterogeneous graph construction, hybrid analytical modeling, federated learning coordination, and compliance monitoring within a unified platform for effective Trade-Based Money Laundering detection and institutional oversight.

4.1.1 Programming Language and Framework Selection

The implementation primarily uses Python for backend services and model training (PyTorch + PyTorch Geometric — e.g., GATv2Conv for graph attention), Flower for federated orchestration (custom FedAvg strategy), and Apache Kafka for durable streaming. Frontend and UI services use Next.js, TypeScript and Tailwind; FastAPI provides REST endpoints. Memgraph stores heterogeneous transaction graphs and PostgreSQL holds audit logs and STR records. Services are containerized with Docker for reproducible deployment.

4.1.2 Platform Selection

The proposed TBML detection framework is developed within a Windows-based environment consisting of interconnected frontend monitoring services and backend analytical infrastructures. The frontend platform manages authentication workflows, dashboard visualization, STR generation, regulatory monitoring, and institutional transaction management, while the backend analytical environment performs transaction streaming, graph generation, fraud inference, and federated model synchronization operations.

The implementation utilizes Docker-based isolated runtime environments for Kafka services, graph databases, backend APIs, and federated learning modules to improve deployment consistency and distributed service orchestration. Memgraph is utilized for graph relationship management and transactional connectivity analysis, whereas PostgreSQL functions as the centralized synchronization database between analytical services and dashboard environments. Visual Studio Code and GitHub are utilized for collaborative development, source-code management, debugging, and version control operations throughout the implementation lifecycle.

4.1.3 Code Conventions

The codebase follows concise, practical conventions to improve readability and maintenance. Python modules and functions use `snake_case`; classes and UI components use `PascalCase`. Examples of graph entities and relationships include `(:Account)`, `(:Transaction)`, `(:Document)` and `[:SENDS]`, `[:TO]`, `[:HAS_DOC]`. REST endpoints follow clear, resource-oriented naming (e.g., `/api/transactions/{id}/score`). Configuration is managed via `.env` files, logging uses structured JSON logs, and type hints plus docstrings are used where helpful.

Naming Conventions

Naming conventions within the proposed framework are designed to remain semantically meaningful and aligned with domain-specific financial monitoring operations. Python variables, backend utility functions, Kafka services, and transaction-processing modules follow `snake_case` conventions, whereas analytical classes, federated learning modules, and frontend components utilize `PascalCase` notation. REST API endpoints, transaction services, and graph-processing utilities are named according to their operational functionality to improve implementation readability and maintainability.

Graph entities, transactional relationships, and database schemas additionally follow consistent naming conventions across graph and relational database environments. Examples include graph entities such as `(:Account)`, `(:Transaction)`, and `(:Document)`, along with transactional relationships including `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]` utilized throughout graph construction and fraud analysis operations.

4.1.4 Difficulties Encountered and Strategies Used to Tackle

Implementation challenges included noisy and imbalanced labels, large graph sizes and memory constraints, streaming latency, and privacy-preserving coordination across institutional clients. Mitigations applied in the implementation include class rebalancing and targeted data augmentation for scarce labels, subgraph sampling and mini-batching to control memory, Kafka buffering and asynchronous consumers to reduce end-to-end latency, and Flower with secure aggregation and containerized deployments (Docker) to isolate client environments while enabling federated training.

4.1.5 System Implementation

The system implementation of the proposed TBML detection framework follows a distributed event-driven architecture integrating Kafka-based transaction streaming, heterogeneous graph generation, Hybrid HGNN–LSTM analytical processing, federated collaborative learning, real-time fraud inference, and dashboard-driven compliance monitoring. Backend analytical services, graph databases, inference pipelines, and frontend monitoring environments communicate through synchronized APIs, Kafka streams, and centralized PostgreSQL audit infrastructures to support scalable real-time Trade-Based Money Laundering detection and institutional monitoring operations.

4.1.6 Methodology of Synthetic Dataset Synthesis

The synthetic dataset was designed to mirror the complexity of real-world trade finance as closely as possible. It begins with a baseline network of financial entities, including Accounts, Documents, and Transactions, generated using power-law distributions so that the transaction volume looks more like an actual institutional environment. From there, the pipeline introduces temporal variation, commodity-specific price anomalies, and multi-currency layering patterns to create training data that is both structured and realistic.

The implementation follows a modular pipeline built with Pandas for data preparation

and Memgraph for graph-based storage. Each transaction is linked to its corresponding trade document through a schema-consistent mapping so that the relationships needed for Graph Neural Network (GNN) feature extraction are preserved. With temporal constraints and controlled noise injection, the pipeline produces a broad range of behaviors, from ordinary legitimate trades to carefully concealed, high-velocity laundering patterns.

Adversarial Refinement for Dataset Fidelity

To make the synthetic data behave more like real financial activity, the generation pipeline was refined with a set of adversarial and statistically grounded adjustments **synthetic-tbml-framework**. The goal was to help the fraud-detection framework learn meaningful transactional and structural patterns instead of depending on easy shortcuts or deterministic signals.

1. **Price Multiplier Dynamics:** Instead of using fixed pricing rules, the pipeline now uses Gaussian Mixture Models (GMMs) and stochastic blending so that some fraudulent trades look closer to genuine market behavior and are harder to catch with simple thresholds.
2. **Elimination of Feature Bias:** The *port_log_status* attribute was removed to prevent label leakage and push the Graph Neural Network to learn from relational and structural cues instead.
3. **KYC/Dormancy Engineering:** Overlapping dormancy periods and controlled noise were added to produce customer activity patterns that feel more realistic than a simple dormant-versus-active split.
4. **Universal Topology Injection:** Structures such as Cycles, Fan-In, and Gather-Scatter patterns were spread across all transaction classes so that no single graph shape automatically signals fraud.
5. **Refined Pattern Classification:** Labels such as LEGIT and SUSPICIOUS were removed, and a neutral *No_Pattern* category was added so that structural topology stays separate from fraud labeling.
6. **Temporal Realism:** The dataset was aligned to a realistic one-year operational timeline (Dec 2024–Dec 2025), with transaction durations modeled around practical financial activity windows.

-
7. **Scale-Free Network Topology:** Pareto/Zipf-based node distributions were introduced to simulate high-volume institutional or “Whale” entities that can hide suspicious activity inside dense transaction networks.
 8. **Log-Normal Transaction Amounts:** Transaction values were generated with Log-Normal distributions so they better resemble real financial amounts instead of uniform synthetic values.
 9. **Multi-Hop Structural Integrity:** Rule-based constraints were applied across multi-hop paths to keep the graph consistent and preserve realistic relationship propagation.
 10. **FX Settlement/Currency-Jump Awareness:** Multi-currency flows and FX spread variations were added so the framework could learn cross-border layering and TBML settlement patterns.
 11. **Stochastic Trade Price Blending:** Illicit pricing distributions were blended with legitimate behavior to reduce obvious tabular shortcuts and encourage context-aware inference.
 12. **Data Leakage Mitigation:** Auxiliary indicators that could reveal fraud too directly were removed to improve generalization during training.
 13. **Endurance-Based Simulation:** Multi-hop cycles were modeled as sustained behaviors rather than isolated short-lived events to make the generated graph more realistic.

Kafka-Based Transaction Streaming

The proposed TBML detection framework utilizes Apache Kafka to support asynchronous transaction streaming and event-driven communication between distributed analytical services. A Kafka producer continuously generates synthetic SWIFT transaction payloads containing sender identifiers, receiver identifiers, transaction amounts, commodity information, trade quantities, price deviation scores, and weight-gap attributes before publishing them into the `incoming_swift_messages` topic for downstream analytical processing.

Kafka consumer services continuously subscribe to the transaction stream and process incoming financial events in real time for graph generation and fraud inference operations. The consumer pipeline validates transaction payloads, constructs heterogeneous graph relationships within Memgraph, extracts multi-hop transactional subgraphs, and forwards graph tensors to the Hybrid HGNN–LSTM inference engine for downstream fraud prediction and PostgreSQL audit logging. The streaming architecture enables decoupled communication between frontend services, graph-processing environments, and distributed analytical modules operating within the proposed framework.

4.1.7 Graph Construction and Feature Extraction

The graph construction pipeline transforms streamed financial transactions into heterogeneous graph representations for downstream graph-based analytical processing. Incoming transaction payloads consumed from the `incoming_swift_messages` Kafka topic are processed using Cypher-based graph ingestion operations within Memgraph, where transactional entities and financial relationships are dynamically mapped into interconnected graph structures as illustrated in Figure ??.

The generated graph schema consists of heterogeneous node entities including `(:Account)`, `(:Transaction)`, and `(:Document)` connected through relationships such as `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]`. The feature extraction pipeline subsequently performs constrained multi-hop subgraph traversal around the target transaction to generate node feature tensors, graph edge mappings, sequential transaction representations, and trade metadata tensors required for Hybrid HGNN–LSTM-based fraud inference operations.

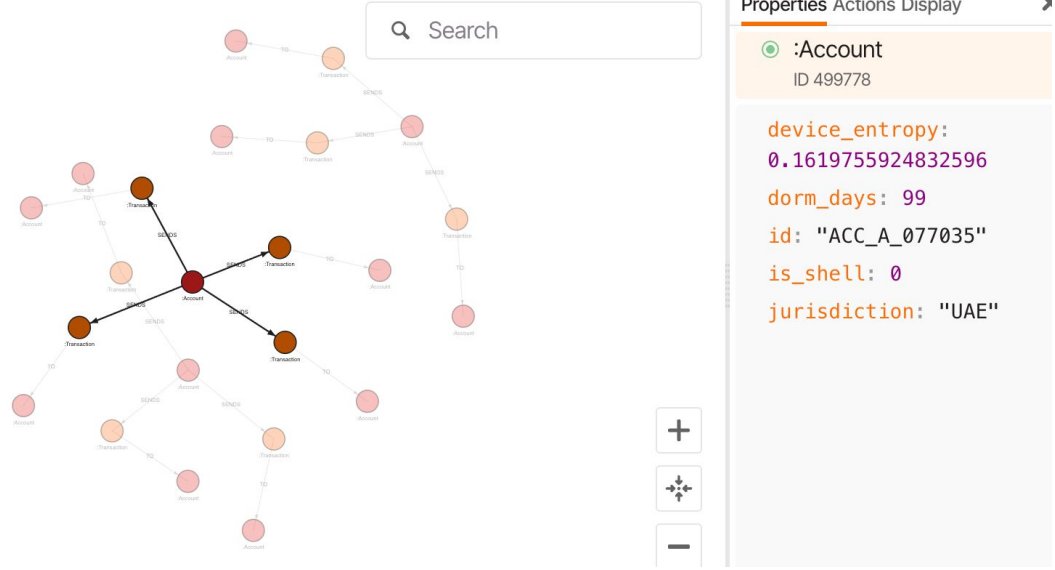


Figure 4.1: Heterogeneous financial transaction graph visualized within the Memgraph Lab environment.

Hybrid HGNN–LSTM Analytical Implementation

The proposed Hybrid HGNN–LSTM analytical module is implemented using PyTorch and PyTorch Geometric to perform graph-driven fraud analysis, temporal transaction modeling, and trade metadata learning for intelligent TBML detection. The implementation receives heterogeneous graph tensors generated from the Memgraph environment, including KYC node attributes, transactional edge relationships, sequential SWIFT transaction records, and trade-deviation features before forwarding them into the analytical inference pipeline. By combining graph attention learning with temporal sequence analysis and trade-feature fusion, the framework is capable of capturing both structural and behavioral fraud patterns across interconnected financial transaction networks.

The graph-learning branch utilizes a three-layer **GATv2Conv** architecture with multi-head attention mechanisms to perform neighborhood aggregation and graph message passing across interconnected financial entities **brody2022gatv2**. The implemented graph layers process KYC-related node attributes such as shell-company indicators, jurisdiction metadata, and dormant-account characteristics to generate enriched graph embeddings capable of exposing hidden intermediary relationships and suspicious transaction-routing behavior. ReLU activation functions and dropout regularization are integrated after each graph convolution layer to improve representation stability during federated training operations. The generated graph embeddings are then aggregated using `global_mean_pool()` and fused with sequential SWIFT transaction features before being processed through the

LSTM branch for temporal anomaly detection and sequential transaction behavior modeling. The temporal branch captures repeated payment structures, abnormal transaction timing behavior, and cyclic financial movement patterns that are difficult to identify using standalone graph-learning approaches.

Trade-specific metadata such as price-deviation attributes and cargo weight-gap indicators are independently processed using multilayer perceptron operations consisting of fully connected layers, Layer Normalization, ReLU activations, and dropout regularization mechanisms. The generated graph embeddings, temporal sequence representations, and trade-feature vectors are fused within the final analytical layer before being forwarded into the dense classification module for fraud prediction. The final classification layer categorizes transactions into predefined classes including CLEAN, WATCHLIST, and FRAUD. Experimental evaluation demonstrated stable federated convergence and strong fraud-class Precision–Recall performance across distributed institutional datasets. The overall implementation workflow of the proposed Hybrid HGNN–LSTM analytical module is illustrated in Figure ??.

```

1 # =====
2 # HYBRID HGNN-LSTM IMPLEMENTATION
3 # =====
4
5 # --- Graph Attention Layers ---
6 # Multi-hop neighborhood aggregation
7 self.gat1 = GATv2Conv(
8     in_channels=kyc_dim, out_channels=hidden_dim, heads=2, concat=False
9 )
10
11 self.gat2 = GATv2Conv(
12     in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
13 )
14
15 self.gat3 = GATv2Conv(
16     in_channels=hidden_dim, out_channels=hidden_dim, heads=2, concat=False
17 )
18
19 # --- Temporal Sequence Branch ---
20 # Sequential SWIFT transaction modeling
21 self.lstm = nn.LSTM(
22     input_size=hidden_dim + swift_dim, hidden_size=lstm_hidden, batch_first=True
23 )
24
25 # =====
26 # GRAPH MESSAGE PASSING
27 # =====
28
29 x = F.dropout(F.relu(self.gat1(kyc_x, edge_index)), p=0.2, training=self.training)
30
31 x = F.dropout(F.relu(self.gat2(x, edge_index)), p=0.2, training=self.training)
32
33 enriched_nodes = F.dropout(
34     F.relu(self.gat3(x, edge_index)), p=0.2, training=self.training
35 )

```

```

1 # =====
2 # GLOBAL GRAPH POOLING
3 # =====
4
5 graph_context = global_mean_pool(enriched_nodes, batch).unsqueeze(1)
6
7 # =====
8 # GRAPH + TEMPORAL FEATURE FUSION
9 # =====
10
11 graph_context_expanded = graph_context.expand(seq_data.size(0), seq_data.size(1), -1)
12
13 combined_seq = torch.cat([seq_data, graph_context_expanded], dim=-1)
14
15 # Temporal anomaly detection
16 lstm_out, _ = self.lstm(combined_seq)
17
18 # =====
19 # TRADE FEATURE PROCESSING
20 # =====
21
22 trade_emb = self.trade_mlp(trade_features)
23
24 # =====
25 # FINAL FRAUD CLASSIFICATION
26 # =====
27
28 fused_vector = torch.cat([lstm_out[-1], trade_emb], dim=1)
29
30 return self.classifier(fused_vector)

```

Figure 4.2: Implementation workflow of the proposed Hybrid HGNN–LSTM analytical module.

Federated Learning Implementation

The federated learning environment is implemented using the Flower federated learning framework to support distributed privacy-preserving fraud detection across multiple institutional clients. The implementation utilizes a centralized aggregation server and

three participating institutional clients, where each client performs localized training of the proposed Hybrid HGNN–LSTM analytical model using institution-specific transaction graphs, sequential SWIFT transaction representations, and fraud classification labels without directly sharing sensitive financial records outside the local analytical environment.

The centralized aggregation server implements a custom federated strategy by extending the `flwr.server.strategy.FedAvg` class and overriding the `aggregate_fit()` operation for distributed parameter aggregation. During each federated communication round, locally trained model weights generated from participating institutional clients are transmitted to the aggregation server where the **FedAvg** algorithm performs parameter averaging across synchronized client updates to generate a globally optimized fraud detection model. The server initializes the Hybrid HGNN–LSTM analytical architecture using synchronized feature dimensions including three-dimensional KYC node features, one-dimensional SWIFT transaction attributes, two-dimensional trade metadata features, hidden embedding dimensions of size 32, and three-class fraud prediction outputs.

The federated implementation performs ten synchronized communication rounds using complete client participation configurations including `fraction_fit=1.0`, `min_fit_clients=3`, and `min_available_clients=3` to maintain stable distributed convergence across institutional environments **kennedy2025federated**. After the final aggregation round, the globally optimized model parameters are converted from Flower parameter streams into NumPy tensors using `parameters_to_ndarrays()` before being serialized and stored as `global_model.npz` for downstream real-time fraud inference operations. The implemented federated workflow enabled collaborative fraud learning, stable distributed convergence, and privacy-preserving analytical synchronization while maintaining complete institutional data isolation across participating financial organizations.

4.1.8 User Interface and Compliance Workflow Implementation

The user interface and compliance-monitoring environment is implemented using Next.js, TypeScript, Tailwind CSS, Prisma ORM, Zustand state management, and PostgreSQL to support secure institutional fraud monitoring and centralized regulatory investigation workflows. The frontend architecture follows a modular service-oriented design consisting of protected dashboard routes, middleware-based authentication guards, API handlers, Prisma-powered database interaction layers, and role-specific monitoring inter-

faces integrated with the federated analytical inference environment.

Authentication and Role Validation Pipeline

The authentication workflow is implemented using JWT-based session authorization combined with centralized middleware validation and protected route interception mechanisms. Incoming requests targeting protected dashboard routes are validated through middleware guards where session tokens are verified against predefined authorization scopes including `BANK_ADMIN` and `REGULATOR`. Once validated, the session context is propagated across frontend environments through centralized authentication providers and Zustand-managed session states before protected dashboard components and compliance services are rendered within the client environment.

Transaction Monitoring and Dashboard Rendering

Fraud predictions generated by the federated Hybrid HGNN–LSTM inference pipeline are synchronized into centralized PostgreSQL audit tables through backend transaction-processing services. Protected Next.js API handlers subsequently retrieve transaction classifications, fraud scores, STR metadata, and institutional monitoring information using Prisma query operations before dynamically rendering analytical outputs within role-specific dashboard interfaces. The banking administrator dashboard supports transaction filtering, fraud investigation workflows, watchlist escalation procedures, and false-positive reclassification operations through synchronized API-driven transaction state updates across centralized audit environments.

STR Generation and Notification Workflow

Transactions classified as fraudulent trigger Suspicious Transaction Report (STR) generation workflows implemented through secured API endpoints and PostgreSQL-backed audit synchronization services. Institutional administrators initiate STR generation requests through protected dashboard interfaces where transaction metadata, fraud classifications, investigation summaries, institutional remarks, and compliance status information are serialized and stored within centralized STR audit tables. The generated STR records are subsequently propagated into regulator-facing monitoring environments through synchronized notification services and backend compliance-routing workflows.

Regulatory Investigation and Compliance Monitoring

The regulatory monitoring environment consumes institution-generated STR records, fraud-classified transaction histories, compliance metadata, and supporting analytical evidence through protected API routes and Prisma-based database interaction layers. Regulatory authorities can perform investigation workflows including document-verification requests, institutional inquiry procedures, transaction review operations, and account-freeze initiation requests through centralized compliance interfaces synchronized with backend audit environments. The implemented monitoring workflow supports synchronized fraud prediction, institutional verification, STR lifecycle management, and centralized compliance monitoring within a unified real-time TBML analytical platform.

4.2 Testing and Validation

This section provides a comprehensive overview of the testing and validation processes employed to ensure the correctness, reliability, and performance of the proposed TBML detection framework. It includes descriptions of the testing environment, unit testing strategies for individual components, integration testing for end-to-end workflow validation, performance evaluation under simulated transaction loads, and security assessments to validate access control mechanisms and data protection measures within the framework.

4.2.1 Testing Environment

The testing process was conducted in a controlled development environment consisting of Python 3.11, PyTorch, PyTorch Geometric, Apache Kafka, Memgraph, PostgreSQL, and Flower federated learning services operating within isolated virtual environments created using `venv`. Frontend monitoring interfaces were implemented and validated using Next.js, TypeScript, Tailwind CSS, Prisma ORM, Zustand state management, and JWT-based authentication workflows.

The testing workflow additionally utilized Playwright-based browser automation for validating protected dashboard routes, institutional transaction workflows, STR generation pipelines, and compliance-monitoring interfaces. Database synchronization latency, transaction propagation consistency, backend API communication, and distributed fraud-inference workflows were validated under controlled institutional transaction loads to ensure reliable execution across distributed analytical environments.

4.2.2 Unit Testing

Each backend analytical service, dashboard workflow, authentication mechanism, and database synchronization component was independently tested to validate correctness, boundary handling, synchronization stability, and operational consistency before integration into the complete TBML detection framework.

Unit Testing of Authentication and RBAC Module

The authentication and role-based access control environment was independently validated to ensure secure user authentication, authorization management, middleware-driven route protection, credential verification, and institutional role isolation across banking and regulatory dashboard interfaces. The testing workflow additionally verified protected API access, invalid credential handling, unauthorized route restrictions, and role-specific dashboard rendering across distributed monitoring environments.

Table 4.1: TC-001: Unit Testing of Authentication and RBAC Module

Serial Number of Test Case	TC-001
Name of Test Case	Protected Dashboard Access Validation
Feature being Tested	Authentication and Authorization Validation
Description	Validate protected route access, JWT verification, and role-based authorization
Sample Input	Invalid JWT session and unauthorized user request
Expected Output	Unauthorized access denied with authentication error response
Actual Output	Invalid sessions blocked and unauthorized users redirected successfully
Remarks	Successful test

Middleware-driven session validation successfully enforced protected route isolation and role-specific dashboard authorization across institutional monitoring environments. The testing workflow validated JWT token verification, session-expiration handling, credential validation, protected API access control, and middleware-based authorization checks across banking and regulator dashboards. Invalid credentials, unauthorized API requests, expired authentication sessions, and direct access attempts to protected routes were correctly rejected using backend access-control policies before protected resources

were rendered. The authentication workflow additionally verified successful role mapping for `BANK_USER` and `ADMIN` entities to ensure secure dashboard isolation and controlled access to institutional fraud-monitoring operations.

Unit Testing of Kafka Transaction Streaming

The Kafka transaction streaming environment was independently tested to validate asynchronous transaction publishing, real-time event synchronization, payload integrity validation, and downstream analytical communication across distributed backend services. The testing workflow additionally verified message delivery consistency, topic-based transaction propagation, malformed payload handling, and reliable synchronization between Kafka producer and consumer services operating within the proposed framework.

Table 4.2: TC-002: Unit Testing of Kafka Transaction Streaming

Serial Number of Test Case	TC-002
Name of Test Case	Publish Valid SWIFT Transaction
Feature being Tested	Kafka Event Streaming and Message Synchronization
Description	Validate transaction publishing and topic-based event synchronization
Sample Input	JSON transaction payload containing sender, receiver, amount, and trade metadata
Expected Output	Transaction published into <code>incoming_swift_messages</code> topic successfully
Actual Output	Transaction streamed and synchronized successfully across Kafka services
Remarks	Successful test

The Kafka streaming module successfully validated asynchronous transaction publishing and backend event synchronization across distributed analytical services. Producer and consumer services maintained reliable message propagation and transaction consistency during continuous event-stream processing operations. Invalid payload structures, malformed transaction metadata, and incomplete financial events were correctly rejected through backend validation workflows before downstream graph-processing and fraud-inference operations were initiated.

Unit Testing of Graph Construction Module

The heterogeneous graph-construction environment was independently tested to validate transactional node generation, relationship mapping, multi-entity graph synchronization, and Cypher-based ingestion workflows within the Memgraph analytical environment. The testing workflow additionally verified graph consistency, entity-link generation, transaction propagation accuracy, and downstream graph availability for Hybrid HGNN-LSTM analytical inference operations.

Table 4.3: TC-003: Unit Testing of Graph Construction Module

Serial Number of Test Case	TC-003
Name of Test Case	Heterogeneous Graph Entity Generation
Feature being Tested	Memgraph Graph Construction and Relationship Mapping
Description	Validate transactional node generation and graph relationship synchronization
Sample Input	Valid SWIFT transaction payload with account, trade, and document metadata
Expected Output	Graph nodes and transactional relationships created successfully within Memgraph
Actual Output	Heterogeneous graph entities and multi-hop relationships generated successfully
Remarks	Successful test

The graph-construction workflow successfully generated heterogeneous transactional graphs within the Memgraph environment using Cypher-based ingestion and relationship-mapping operations. Graph entities including `(:Account)`, `(:Transaction)`, and `(:Document)` were correctly synchronized with transactional relationships such as `[:SENDS]`, `[:TO]`, and `[:HAS_DOC]` during continuous event-stream processing. The testing workflow additionally validated graph consistency, entity-link integrity, and multi-hop transactional connectivity required for downstream subgraph extraction and graph-based fraud inference operations within the proposed framework.

Unit Testing of STR Generation Workflow

The STR generation workflow was independently tested to validate suspicious transaction report generation, compliance-record synchronization, fraud-case escalation, and

regulator-side notification propagation across distributed institutional monitoring environments. The testing workflow additionally verified report persistence, transaction-metadata preservation, audit consistency, and downstream synchronization between banking dashboards and regulator-facing compliance interfaces.

Table 4.4: TC-004: Unit Testing of STR Generation Workflow

Serial Number of Test Case	TC-004
Name of Test Case	Suspicious Transaction Report Generation
Feature being Tested	STR Generation and Compliance Synchronization
Description	Validate fraud escalation and regulator-side STR synchronization workflow
Sample Input	Fraud-classified transaction selected from institutional monitoring dashboard
Expected Output	STR record generated and synchronized into regulator compliance environment
Actual Output	STR generated, persisted, and synchronized successfully across monitoring services
Remarks	Successful test

The STR workflow successfully validated automated fraud-report generation, PostgreSQL audit synchronization, and regulator-side notification propagation across distributed institutional environments. Generated STR records correctly preserved transaction metadata, fraud classifications, institutional identifiers, and investigation references during downstream compliance synchronization operations. The testing workflow additionally verified successful dashboard synchronization, report persistence, and controlled propagation of suspicious transaction alerts between banking institutions and regulatory monitoring interfaces operating within the proposed framework.

Unit Testing of Notification Workflow

The notification synchronization environment was independently tested to validate fraud-alert propagation, institutional notification delivery, compliance-event synchronization, and regulator-side alert visibility across distributed monitoring interfaces. The testing workflow additionally verified notification persistence, dashboard update consistency, backend event propagation, and synchronization between banking and regulatory com-

pliance environments during fraud escalation operations.

Table 4.5: TC-005: Unit Testing of Notification Workflow

Serial Number of Test Case	TC-005
Name of Test Case	Fraud Alert and Compliance Notification Propagation
Feature being Tested	Notification Synchronization and Alert Delivery
Description	Validate fraud-alert propagation across banking and regulator monitoring environments
Sample Input	Generated STR record linked to fraud-classified transaction
Expected Output	Compliance alerts synchronized into regulator-facing dashboard interfaces
Actual Output	Fraud notifications propagated and synchronized successfully
Remarks	Successful test

The notification workflow successfully validated fraud-alert propagation, institutional investigation updates, and STR lifecycle synchronization across distributed compliance-monitoring environments. Backend event-processing services maintained reliable notification delivery and dashboard synchronization during fraud escalation operations between banking institutions and regulatory authorities. The testing workflow additionally verified notification persistence, regulator-side alert visibility, and consistent compliance-state propagation across protected monitoring interfaces operating within the proposed framework.

4.2.3 Integration Testing

Integration testing was conducted to validate the interaction, synchronization consistency, and workflow stability between distributed analytical services, graph-processing environments, fraud inference pipelines, dashboard-rendering systems, and regulatory compliance workflows operating within the proposed TBML detection framework.

Integration Testing of Kafka and Memgraph Pipeline

This integration test validated end-to-end synchronization between Kafka-based transaction streaming services and the heterogeneous graph-construction environment oper-

ating within Memgraph. The testing workflow additionally verified event-stream consistency, transaction propagation reliability, graph-ingestion stability, and downstream synchronization between distributed backend services.

Table 4.6: TC-006: Integration Testing of Kafka and Memgraph Pipeline

Serial Number of Test Case	TC-006
Name of Test Case	Transaction Stream and Graph Synchronization Workflow
Feature being Tested	Kafka Event Propagation and Memgraph Synchronization
Description	Validate transaction-stream synchronization and heterogeneous graph generation
Sample Input	Valid Kafka transaction payload with trade meta-data and KYC attributes
Expected Output	Transactional graph entities and relationships generated successfully within Memgraph
Actual Output	Kafka events synchronized and heterogeneous graph generated successfully
Remarks	Successful test

The integration workflow successfully validated asynchronous transaction propagation between Kafka producer-consumer services and Memgraph graph-ingestion environments. Streamed SWIFT transaction events were reliably synchronized into graph-processing pipelines without transactional inconsistency or event-loss during continuous backend processing operations. The testing workflow additionally verified stable graph synchronization, multi-entity relationship generation, and downstream availability of transactional subgraphs required for graph-based analytical inference within the proposed framework.

Integration Testing of Fraud Prediction and Dashboard Synchronization

This integration test checked how well the fraud-prediction services, PostgreSQL audit layer, and role-based monitoring dashboards worked together within the proposed TBML framework. It also confirmed that transaction results were stored correctly, updated reliably on the dashboard, and rendered in real time for fraud-monitoring review.

Table 4.7: TC-007: Integration Testing of Fraud Prediction and Dashboard Synchronization

Serial Number of Test Case	TC-007
Name of Test Case	Fraud Prediction and Dashboard Rendering Workflow
Feature being Tested	PostgreSQL Synchronization and Dashboard Visualization
Description	Validate fraud-prediction synchronization into institutional monitoring dashboards
Sample Input	Fraud-classified transaction generated from analytical inference pipeline
Expected Output	Fraud transaction persisted and rendered successfully within dashboard interfaces
Actual Output	Dashboard synchronization completed successfully with valid fraud-state rendering
Remarks	Successful test

Fraud-prediction records, transaction metadata, and institutional risk classifications were synced into the central PostgreSQL audit environment and then displayed through protected dashboard interfaces. The dashboard updates stayed consistent across institutional monitoring services, giving users a clear real-time view of transaction status and fraud outcomes. The test also confirmed stable backend persistence, reliable dashboard refreshes, and secure rendering of fraud-monitoring data for each role-based compliance environment.

Integration Testing of STR Generation Workflow

This integration test covered the full path from institutional fraud review to STR generation, regulator synchronization, and compliance notifications across distributed monitoring environments. It verified that fraud escalation behaved consistently, reports were stored correctly, and regulators could see the same case information across their compliance pipelines.

The STR workflow worked as expected: once a transaction was marked for escalation, the system generated the compliance report, synchronized it to the regulator-side dashboard, and propagated the related notifications across the monitoring environment. Each STR record retained the key fraud classification details, investigation notes, institu-

tional identifiers, and audit references needed for downstream compliance processing. The test also confirmed that compliance alerts reached the relevant banking and regulatory interfaces in a synchronized way.

Table 4.8: TC-008: Integration Testing of STR Generation Workflow

Serial Number of Test Case	TC-008
Name of Test Case	Fraud Escalation and STR Compliance Workflow
Feature being Tested	STR Lifecycle Management and Compliance Synchronization
Description	Validate fraud escalation and regulator-side STR synchronization workflow
Sample Input	Fraud transaction escalated by institutional administrator
Expected Output	STR generated and synchronized successfully within regulator compliance dashboard
Actual Output	Fraud escalation and STR synchronization completed successfully
Remarks	Successful test

Integration Testing of Authentication and Protected Dashboard Rendering

This integration test validated synchronization between JWT-based authentication services, middleware-driven authorization guards, protected API routes, and role-specific dashboard-rendering environments operating within the proposed framework. The testing workflow additionally verified secure session propagation, authorization consistency, and protected resource accessibility across institutional monitoring interfaces.

Middleware-driven authentication workflows successfully validated session authorization, protected route synchronization, secure API accessibility, and institutional role isolation across distributed monitoring interfaces. Dashboard-rendering operations correctly enforced role-specific access-control policies for `BANK_USER`, `ADMIN`, and `REGULATOR` entities during backend synchronization workflows. The testing workflow additionally verified invalid-session rejection, unauthorized route restrictions, and secure propagation of authenticated dashboard resources across institutional compliance-monitoring environments.

Table 4.9: TC-009: Integration Testing of Authentication and Protected Dashboard Rendering

Serial Number of Test Case	TC-009
Name of Test Case	Authenticated Dashboard Access and Rendering Workflow
Feature being Tested	Session Validation and Role-Based Dashboard Authorization
Description	Validate authenticated dashboard rendering and protected route authorization
Sample Input	Valid JWT-authenticated institutional user session
Expected Output	Authorized dashboard rendered successfully based on institutional role
Actual Output	Protected dashboard rendered successfully with valid role authorization
Remarks	Successful test

4.2.4 System Testing

System testing was used to check how the proposed TBML detection framework behaved under realistic operational load. The focus was on end-to-end browser validation, protected dashboard execution, concurrent PostgreSQL access, API response times, and latency behavior across the fraud-monitoring and compliance-management services. The goal was to confirm that the system remained stable, responsive, and consistent when several components were active at the same time in a distributed institutional setting.

Browser Automation and Workflow Validation

Browser-based end-to-end validation was performed using Playwright automation workflows executed within Chromium testing environments. The testing workflow validated protected dashboard rendering, role-based authentication, STR lifecycle execution, fraud-notification propagation, compliance-action synchronization, and institutional transaction visibility restrictions across distributed frontend and backend services. All automated workflow executions completed successfully without unauthorized access, synchronization failure, or dashboard inconsistency during operational validation.

Table 4.10: TC-010: Browser Automation and Workflow Validation Results

Workflow Validation	Status
Admin Authentication and Dashboard Rendering	Successful
Fraud Case Retrieval and STR Visibility	Successful
GET_DOCUMENTS Compliance Workflow	Successful
FREEZE_ACCOUNT Regulatory Workflow	Successful
Fraud Verdict Marking Workflow	Successful
Bank User STR Generation Workflow	Successful
Fraud Notification Propagation Workflow	Successful
Protected Route and RBAC Validation	Successful
Institutional Transaction Visibility Restriction	Successful
Sign-In Page Smoke Testing	Successful

The Playwright automation workflow confirmed that institutional fraud-monitoring actions could be carried out in sequence across protected dashboard environments without interruption. It also verified secure session persistence, consistent API synchronization, proper workflow-level authorization, and stable dashboard rendering for both **ADMIN** and **BANK_USER** users. In addition, the test showed that STR generation, fraud escalation, regulator-side compliance synchronization, and notification propagation all worked correctly in the automated end-to-end workflow.

Concurrent Database Scalability and API Performance Analysis

Concurrent database scalability testing assessed how well PostgreSQL synchronization held up under increasing institutional user loads, along with dashboard query performance, fraud-filtering latency, and STR retrieval consistency. The benchmark used Prisma ORM and Next.js backend services, and each simulated user sent five continuous transactional requests while accessing the dashboard. To keep the workload stable, the test used batched asynchronous requests, pooled PostgreSQL connections, and a limit of 15 active connections so that synchronization stayed reliable and backend resources were not overloaded during heavy monitoring activity.

Latency benchmarking was carried out by recording request-start and response-finish timestamps during concurrent API execution. For each request, the workflow captured the start time using `startTime = performance.now()`, sent the asynchronous API call, and then recorded the end time using `endTime = performance.now()` once the response returned. The latency was calculated as `latency = endTime - startTime`. The workload was run in parallel so that multiple simulated institutional users could issue dashboard and compliance queries at the same time under controlled concurrency limits.

Average latency reflected the mean response time across all successful requests, while P95 latency captured the slower end of the response distribution for 95% of requests during peak load. The benchmark also checked request throughput, API synchronization consistency, transaction persistence reliability, and stable PostgreSQL query execution across distributed compliance-monitoring environments.

The benchmarking metrics were computed using the following operational latency equations:

$$L_{\text{avg}} = \frac{1}{N} \sum_{i=1}^N (T_{\text{response}_i} - T_{\text{request}_i}) \quad (4.1)$$

$$L_{95} = \text{Percentile}_{95}(L_1, L_2, \dots, L_N) \quad (4.2)$$

Table 4.11: TC-011: Dashboard Transaction Retrieval Scalability Analysis

Concurrent Users	Total Re-quests	Avg Latency (ms)	P95 Latency (ms)	Success Rate
10	50	297.33	884.28	100%
50	250	156.05	174.75	100%
100	500	159.99	181.30	100%
250	1250	158.53	176.99	100%
500	2500	162.14	180.72	100%

The dashboard transaction retrieval workflow demonstrated stable query scalability and consistent dashboard-rendering performance across increasing institutional user loads. Average latency values remained nearly constant between 50 and 500 concurrent users, indicating efficient PostgreSQL query execution, optimized transactional indexing, and stable backend synchronization during high-frequency dashboard-access operations.

The relatively low variation between average and P95 latency measurements further indicated predictable response behavior and controlled query-execution overhead under concurrent transactional workloads.

Table 4.12: TC-012: Fraud Transaction Filtering Scalability Analysis

Concurrent Users	Total Re-quests	Avg Latency (ms)	P95 Latency (ms)	Success Rate
10	50	159.58	174.04	100%
50	250	158.15	181.58	100%
100	500	147.56	170.37	100%
250	1250	148.18	164.58	100%
500	2500	263.80	731.88	100%

The fraud-transaction filtering workflow demonstrated efficient analytical-query execution and stable institution-specific transaction retrieval during concurrent dashboard operations. Lower average latency values under moderate workloads indicated optimized fraud-state filtering and indexed transactional query execution across backend analytical services. Increased P95 latency under 500-user workloads indicated temporary synchronization overhead during peak concurrent analytical filtering operations; however, the framework maintained complete request reliability, stable API synchronization, and consistent fraud-state propagation throughout distributed monitoring workflows.

Table 4.13: TC-013: STR Compliance Report Retrieval Scalability Analysis

Concurrent Users	Total Re-quests	Avg Latency (ms)	P95 Latency (ms)	Success Rate
10	50	2694.21	7311.50	100%
50	250	2101.61	4791.52	100%
100	500	856.39	2149.27	100%
250	1250	467.03	600.67	100%
500	2500	465.62	596.76	100%

The STR compliance retrieval workflow showed higher latency than the other dashboard operations because it had to aggregate compliance reports, synchronize transaction history, retrieve regulator-side metadata, and execute several relational queries during report generation. Even so, the drop in average latency as concurrency increased suggested

that PostgreSQL query handling became more stable, pooled connections were used more efficiently, and relational queries were executed more smoothly during sustained processing.

Although compliance-report generation added extra processing overhead, the framework still maintained consistent request behavior and a complete success rate across distributed monitoring environments. Overall, the testing workflows showed reliable back-end synchronization, controlled API responses, stable federated analytical execution, and steady PostgreSQL performance as institutional workload increased.

Taken together, the testing results confirmed that the proposed TBML detection framework remained reliable, stable, and consistent across graph-processing, federated inference, dashboard synchronization, and compliance-management services. The scalability results also showed that PostgreSQL execution stayed stable and latency remained controlled even as institutional traffic increased.

4.2.5 Model Performance Evaluation

The proposed federated Hybrid HGNN–LSTM framework was evaluated across three distributed institutional clients (BANKA, BANKB, and BANKC) to validate fraud-classification capability and federated convergence stability across heterogeneous transactional environments. Multi-class prediction performance for *Clean*, *Watchlist*, and *Fraud* transaction states was analyzed using Precision, Recall, F1-Score, confusion-matrix validation, and Precision–Recall benchmarking. Federated parameter synchronization was performed using the FedAvg aggregation strategy across 10 federated communication rounds.

The transactional dataset was partitioned using a stratified temporal split strategy to preserve chronological separation between training and testing samples while maintaining balanced representation across minority fraud categories. Weighted cross-entropy optimization and gradient clipping mechanisms were integrated to stabilize distributed training and address fraud-class imbalance during federated convergence operations. Precision–Recall evaluation was prioritized over standalone classification accuracy due to the highly imbalanced nature of illicit transactional behavior within institutional financial environments.

Multi-Class Classification Performance

The final federated classification results obtained after 10 federated aggregation rounds across all participating institutional environments are summarized in Table ???. The evaluation was conducted to analyze the framework’s ability to distinguish legitimate, suspicious, and fraudulent transactional behaviors across distributed institutional datasets under federated learning conditions. The obtained results additionally validated stable analytical convergence and consistent fraud-classification performance across all participating banking clients.

The results showed that the model classified the three transaction states consistently across all institutional clients. Clean transactions were identified with near-perfect Precision and Recall, which helped keep false positives low during large-scale monitoring. The Watchlist class also performed well, with Recall remaining above 0.97, showing that the model could reliably flag suspicious activity for institutional review.

Table 4.14: Federated Multi-Class Classification Performance result

Institution	Transaction Class	Precision	Recall	F1-Score
BANKA	Clean (0)	1.00	1.00	1.00
	Watchlist (1)	0.79	0.97	0.87
	Fraud (2)	0.96	0.73	0.83
BANKB	Clean (0)	1.00	1.00	1.00
	Watchlist (1)	0.79	0.99	0.88
	Fraud (2)	0.98	0.73	0.84
BANKC	Clean (0)	1.00	1.00	1.00
	Watchlist (1)	0.79	0.99	0.88
	Fraud (2)	0.98	0.73	0.84

Fraud-class predictions achieved Precision values between 0.96 and 0.98, which indicates that high-risk transactions were identified with very few false fraud alerts. Fraud Recall stayed around 0.73, but the framework still preserved strong fraud-detection capability even with highly imbalanced data and complex multi-hop laundering structures. Overall, the results showed that the federated model generalized well across distributed institutions without requiring sensitive data to be centralized.

Federated Convergence Analysis

Federated convergence was evaluated by tracking Accuracy, Precision, and Recall across successive aggregation rounds. Separate curves were generated for BANKA, BANKB, and BANKC so that the synchronization behavior of each institutional client could be observed independently. This made it possible to confirm that local updates were being aggregated correctly and that the global model was stabilizing as training progressed.

BANKA Federated Convergence

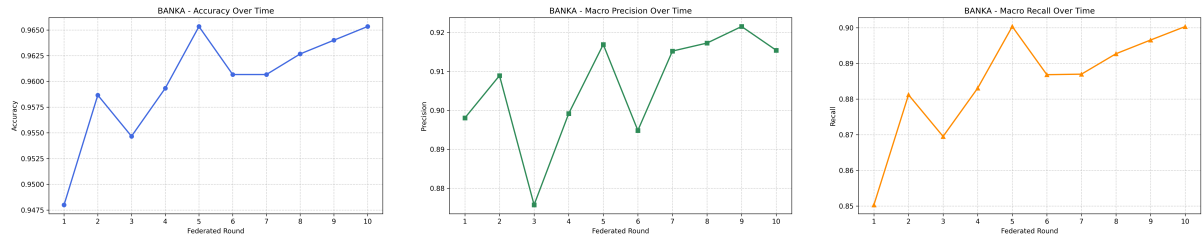


Figure 4.3: BANKA Federated Accuracy, Precision, and Recall Progression Across Federated Aggregation Rounds

As shown in Figure ??, BANKA showed the expected training dynamics for a federated setup. The early variation was reduced as global aggregation and distributed weight synchronization started to dominate the optimization process, and the Accuracy, Precision, and Recall curves became noticeably smoother in the later rounds. That pattern indicates that the client was converging in a controlled way and that the federated updates were being integrated properly.

BANKB Federated Convergence

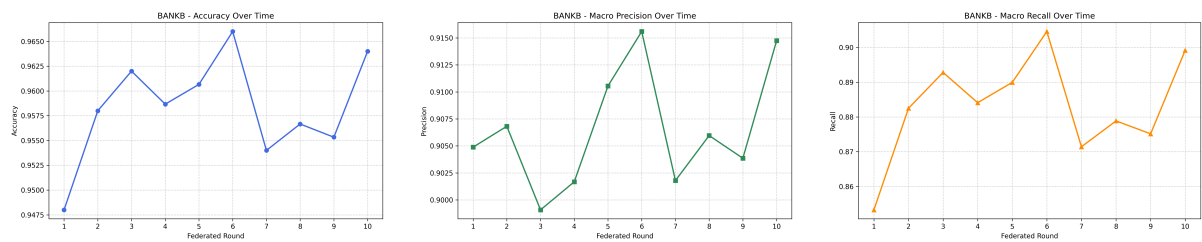


Figure 4.4: BANKB Federated Accuracy, Precision, and Recall Progression Across Federated Aggregation Rounds

As shown in Figure ??, BANKB followed the same stable convergence pattern across the training rounds. Each communication round improved the consistency of the learned parameters and reduced variance in the predictions, which is what we expect when the

global model is being updated from multiple institutional clients. The smoother progression confirms that cross-bank parameter sharing and distributed learning were operating correctly.

BANKC Federated Convergence

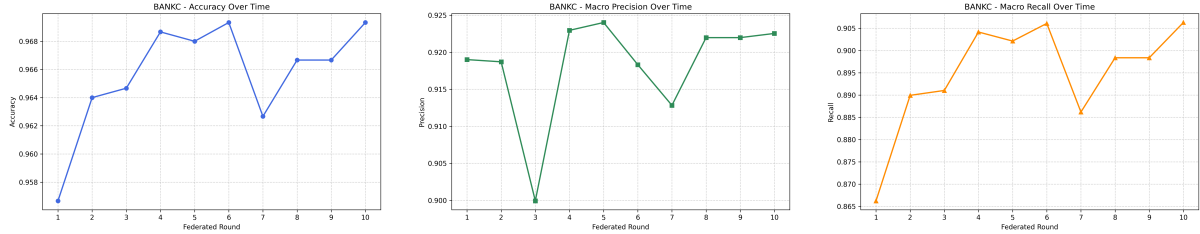


Figure 4.5: BANKC Federated Accuracy, Precision, and Recall Progression Across Federated Aggregation Rounds

As shown in Figure ??, BANKC also converged in a stable and controlled way across the full set of communication rounds. The Accuracy, Precision, and Recall values settled into a consistent range with very little movement in the later stages, which suggests that synchronization was efficient and that the distributed optimization process remained stable. It also indicates that the global model generalized well across different institutional transaction settings.

Precision–Recall and Confusion Matrix Validation

Precision–Recall analysis and confusion-matrix evaluation were used to examine how well the framework separated fraud from legitimate activity, especially for the minority classes where the risk of false positives is higher. Since fraudulent transactions represented only a small portion of the full dataset, Precision–Recall gave a clearer picture of model behavior than accuracy alone.

BANKA Precision–Recall and Confusion Matrix Validation

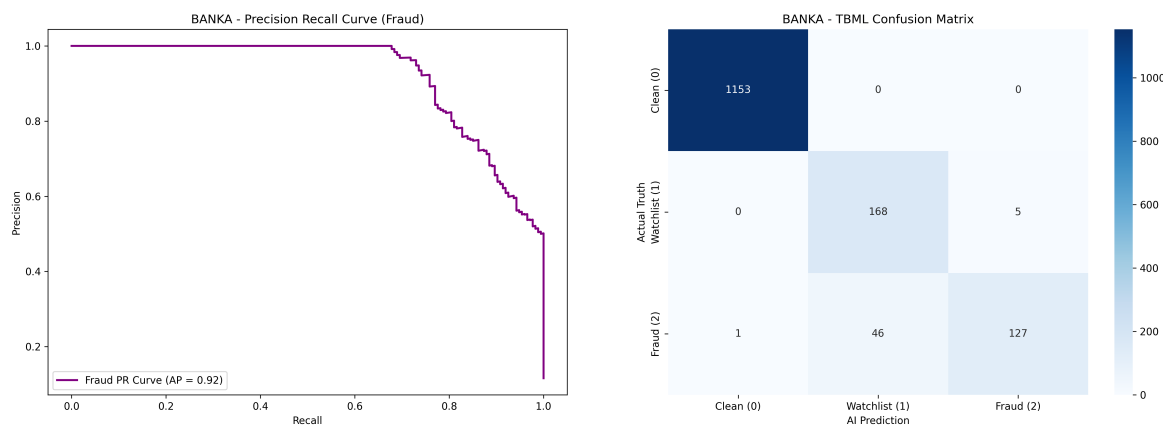


Figure 4.6: BANKA Precision–Recall Curve and Confusion Matrix Validation

As shown in Figure ??, BANKA achieved strong Precision–Recall performance, with Average Precision above 0.90. The curve stayed stable even as recall increased, which shows that suspicious activity was still identified reliably under an imbalanced fraud distribution. The confusion matrix also showed near-perfect classification of legitimate transactions, with very little clean-to-fraud confusion. Most overlap appeared between Watchlist and Fraud, which suggests that the model learned the subtle boundary between those two risky states without producing too many false alarms.

BANKB Precision–Recall and Confusion Matrix Validation

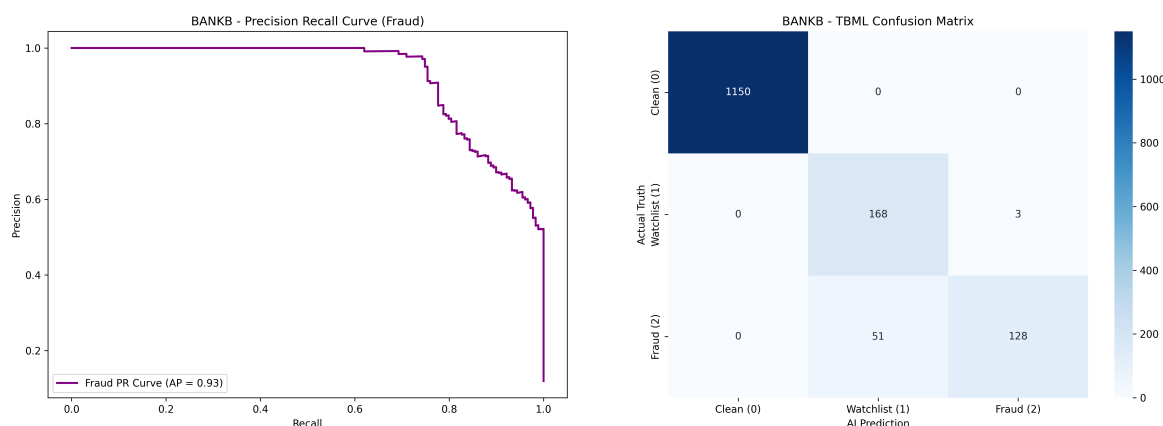


Figure 4.7: BANKB Precision–Recall Curve and Confusion Matrix Validation

As shown in Figure ??, BANKB also delivered stable Precision–Recall behavior, again with Average Precision above 0.90. Precision stayed strong as recall increased, which confirms that the model remained sensitive to fraud without losing control over false positives. The confusion matrix showed that legitimate transactions were classified cleanly,

and only a small number of Fraud cases were mapped to Watchlist, which indicates solid high-risk discrimination during monitoring.

BANKC Precision–Recall and Confusion Matrix Validation

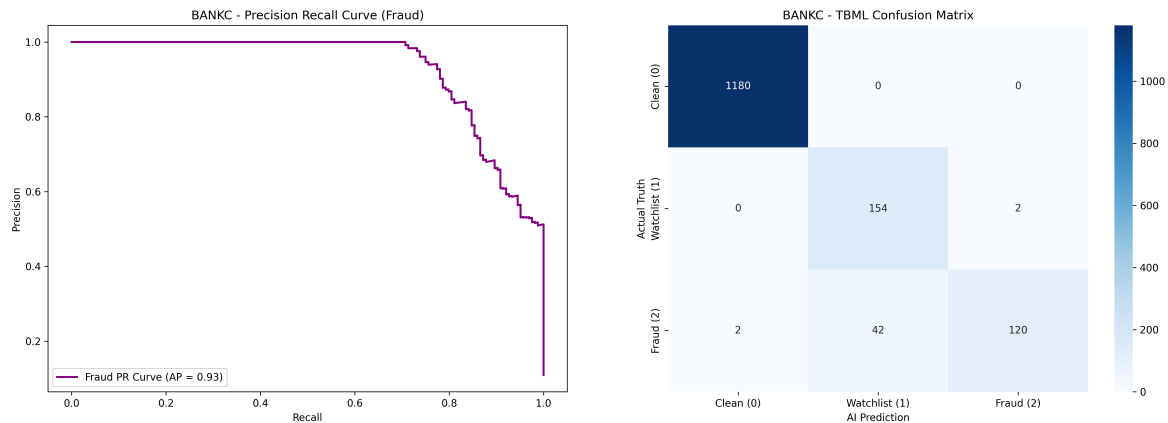


Figure 4.8: BANKC Precision–Recall Curve and Confusion Matrix Validation

As shown in Figure ??, BANKC achieved similarly stable Precision–Recall performance, with Average Precision approaching 0.93. The curve remained steady even at higher recall levels, which points to dependable fraud detection under an imbalanced transaction distribution. The confusion matrix again showed strong consistency for legitimate transactions and very little false-positive spillover into the clean class. Most of the overlap stayed between Watchlist and Fraud, which confirms that the federated model generalized well across isolated institutional datasets.

4.3 Summary

The conducted testing and evaluation workflows validated the operational stability, synchronization reliability, and analytical performance of the proposed TBML detection framework across distributed institutional monitoring environments. The testing results confirmed stable backend communication, reliable transactional synchronization, controlled API response behavior, and successful workflow execution across graph-processing, federated analytical inference, dashboard synchronization, and compliance-management services. Concurrent scalability validation additionally demonstrated stable PostgreSQL query execution and controlled latency characteristics under increasing institutional workloads. The model performance evaluation further validated strong multi-class classification capability, stable federated convergence behavior, and reliable fraud-discrimination performance across heterogeneous institutional transaction environments without requir-

ing centralized sharing of sensitive financial datasets.

Chapter 5

Results and Discussion

CHAPTER 5

RESULTS AND DISCUSSION

This chapter presents the analytical outcomes and operational observations derived from experimental validation of the proposed federated Hybrid HGNN–LSTM framework for Trade-Based Money Laundering detection. Building upon the testing workflows discussed in Chapter 7, the following sections analyze the observed fraud-detection capability, federated learning stability, distributed synchronization behavior, and scalability characteristics of the deployed institutional monitoring framework.

5.1 Overview of Experimental Outcomes

The experimental evaluation demonstrated stable fraud-classification capability, reliable federated convergence behavior, and effective distributed synchronization across heterogeneous institutional transaction environments. The proposed framework successfully maintained multi-class analytical consistency for *Clean*, *Watchlist*, and *Fraud* transaction states while preserving institutional data isolation during federated optimization workflows.

The distributed streaming and graph-processing architecture additionally enabled synchronized real-time transaction propagation, stable heterogeneous graph generation, and controlled dashboard-update behavior during concurrent monitoring workflows. System-level scalability evaluation further demonstrated stable PostgreSQL query execution, efficient transaction persistence, and controlled API response latency under increasing institutional monitoring workloads.

5.2 Operational Performance Outcomes

The deployed monitoring framework demonstrated stable operational performance across distributed fraud-monitoring and compliance-management workflows. Dashboard transaction retrieval, fraud filtering, STR synchronization, and regulator-side compliance propagation maintained reliable execution consistency under concurrent institutional workloads. Table ?? summarizes the observed API-level operational response characteristics across major compliance workflows.

Table 5.1: Operational Workflow Performance Summary

Workflow	Avg Response Time	Operational Observation	Status
Dashboard Transaction Retrieval	~ 160 ms	Stable under concurrent workloads	Successful
Fraud Transaction Filtering	~ 148 ms	Controlled query latency during filtering	Successful
STR Compliance Retrieval	~ 466 ms	Stable relational aggregation behavior	Successful
STR Generation Workflow	< 1 s	Successful compliance synchronization	Successful
Notification Propagation	< 500 ms	Stable regulator-side synchronization	Successful
Federated Fraud Inference	$\sim 1\text{--}2$ s	Stable graph-based analytical prediction	Successful

As observed in Table ??, dashboard synchronization and fraud-filtering workflows maintained comparatively low response latency during concurrent institutional workloads. The STR retrieval workflow exhibited comparatively higher latency due to computationally intensive relational joins, transaction-history aggregation, and compliance-report synchronization operations executed during report-generation workflows. However, stable response consistency and successful request execution were maintained across all monitored operational workflows.

5.3 Graph-Based TBML Detection Capability

The graph-based analytical workflow demonstrated strong capability in learning hidden transactional dependencies and multi-hop entity relationships across heterogeneous institutional transaction environments. Unlike traditional rule-based AML systems that primarily depend on isolated transactional heuristics, the proposed framework leveraged heterogeneous graph construction to capture interconnected relationships between ac-

counts, SWIFT transactions, trade metadata, compliance documents, and institutional entities during fraud-inference workflows.

The confusion-matrix evaluation discussed in Chapter 7 demonstrated that most classification overlap occurred between *Watchlist* and *Fraud* transaction states, while minimal propagation of fraudulent predictions into legitimate transactional categories was observed. This behavior validated the effectiveness of graph-topology learning for identifying suspicious transactional relationships without excessively escalating legitimate institutional transactions into fraud classifications during compliance-monitoring operations.

5.4 Federated Learning Effectiveness

The federated analytical workflow demonstrated stable distributed convergence behavior across all participating institutional clients during iterative FedAvg synchronization operations. The convergence progression analyzed in Chapter 7 showed gradual stabilization of Accuracy, Precision, and Recall metrics during later federated communication rounds, thereby validating effective distributed parameter synchronization and stable institutional knowledge sharing during collaborative optimization workflows.

Table ?? summarizes the observed federated analytical performance across distributed institutional clients after completion of iterative global aggregation workflows.

Table 5.2: Federated Analytical Performance Summary

Institution	Transaction Class	Precision	Recall	F1-Score
BANKA	Fraud (2)	0.96	0.73	0.83
BANKB	Fraud (2)	0.98	0.73	0.84
BANKC	Fraud (2)	0.98	0.73	0.84

As observed in Table ??, the federated Hybrid HGNN–LSTM framework maintained high Fraud-class Precision across all participating institutional environments while preserving stable Recall characteristics during distributed analytical inference. The observed convergence stability additionally indicated that the proposed framework successfully generalized fraud-learning behavior across heterogeneous institutional transaction environments without requiring centralization of sensitive transactional datasets.

5.5 Real-Time Monitoring and Compliance Outcomes

The distributed transaction-monitoring workflow demonstrated stable real-time synchronization of institutional transaction streams across heterogeneous backend analytical services. Kafka-based asynchronous event propagation enabled continuous ingestion of SWIFT transactional payloads into downstream graph-construction and federated inference pipelines without interrupting institutional monitoring workflows.

The automated STR generation and notification workflow additionally improved operational coordination between banking institutions and regulator-facing compliance environments. Fraud-classified transactional events were successfully transformed into structured compliance reports while preserving associated transaction histories, graph metadata, and institutional investigation references during distributed monitoring operations.

5.6 Scalability and Infrastructure Observations

Concurrent scalability evaluation demonstrated stable API response behavior and reliable PostgreSQL synchronization consistency under increasing institutional monitoring workloads. During dashboard transaction retrieval workflows, the framework maintained average response latencies near 156–162 ms across concurrent workloads ranging from 50 to 500 users while sustaining complete request success rates throughout the evaluation process. Similarly, fraud-transaction filtering workflows maintained average response latency near 148–159 ms under moderate concurrent workloads, thereby validating stable pooled-query execution and controlled backend synchronization during real-time monitoring operations.

The STR compliance retrieval workflow exhibited comparatively higher computational latency due to relational joins, transaction-history aggregation, and regulator-side compliance synchronization executed during report-generation workflows. Initial average response latency near 2694 ms during lower concurrent execution stages progressively reduced to nearly 466 ms during higher concurrent workloads, thereby indicating improved query-plan stabilization, PostgreSQL shared-buffer utilization, and optimized relational-query execution during repeated compliance-retrieval operations. Despite occasional tail-latency spikes during high-concurrency execution stages, the infrastructure evaluation maintained stable request consistency and complete request success rates throughout distributed monitoring and compliance-management workflows.

5.7 Summary

The experimental outcomes and analytical observations validated the effectiveness of the proposed federated Hybrid HGNN–LSTM framework for distributed Trade-Based Money Laundering detection and compliance monitoring. The observed results demonstrated stable graph-based fraud inference, reliable federated convergence behavior, controlled scalability characteristics, and effective real-time synchronization across distributed institutional monitoring environments. The framework additionally achieved privacy-preserving collaborative learning without requiring centralization of sensitive transactional datasets, thereby validating its operational suitability for large-scale institutional compliance ecosystems.

Chapter 6

Conclusion

CHAPTER 6

CONCLUSION

The proposed work successfully established a scalable and privacy-preserving federated graph-learning framework for intelligent Trade-Based Money Laundering detection across distributed institutional environments. The developed framework integrated heterogeneous graph construction, federated Hybrid HGNN-LSTM analytical inference, Apache Kafka-based transaction streaming, PostgreSQL-backed compliance orchestration, and regulator-facing monitoring synchronization within a unified operational architecture. Experimental evaluation demonstrated stable fraud-classification capability, reliable federated convergence behavior, and effective distributed synchronization across heterogeneous institutional transaction environments. The deployed framework achieved Fraud-class Precision values approaching 0.98 while maintaining stable Recall characteristics during federated analytical inference workflows. Concurrent scalability evaluation additionally demonstrated stable dashboard retrieval latency near 156–162 ms and controlled fraud-filtering response latency near 148–159 ms during increasing institutional monitoring workloads. The overall analytical and operational outcomes validated the suitability of the proposed framework for scalable, real-time, and privacy-aware TBML detection across distributed institutional compliance ecosystems.

6.1 Limitations

Although the framework performed well during testing, a few practical limitations remain. The current evaluation still depends mostly on synthetically enriched transaction data, since verified real-world TBML datasets are difficult to access. As a result, some newer laundering tactics, cross-border irregularities, and more subtle trade-manipulation patterns may not appear as clearly as they would in live institutional data.

The federated learning pipeline also introduces communication overhead whenever model updates are exchanged and aggregated across participating institutions. That overhead is manageable at the current scale, but it can grow as more banks join the system or as the model is trained on larger transaction graphs. Likewise, very large heterogeneous graphs can increase the cost of multi-hop traversal and distributed inference.

The present implementation is focused on detection, monitoring, and compliance notification. It can flag suspicious activity and support regulator-side review, but it does not

yet carry out direct operational actions such as transaction blocking, account freezing, document locking, or automated compliance-trigger execution. Those functions would need deeper integration with core banking systems and institution-level authorization workflows.

6.2 Future Scope

There are several clear ways to extend the framework into a more production-ready financial crime monitoring system. A useful next step would be to test it on larger, institutionally verified TBML datasets that include real laundering annotations, cross-border trade activity, and changing fraud patterns. That would make the framework more reliable in situations that look closer to actual banking operations.

Future work could also add transformer-based document understanding so the system can read unstructured trade records such as invoices, customs declarations, shipping documents, and letters of credit more effectively. Those semantic features could be combined with the graph-learning pipeline to improve document verification and anomaly detection. The federated setup could also be widened to cover more geographically distributed institutions and a larger decentralized training network.

Another worthwhile direction is stronger privacy protection during federated training. Techniques such as Fully Homomorphic Encryption (FHE), Secure Multi-Party Computation (SMPC), and differential privacy could reduce the risk of gradient leakage and model-inversion attacks. It may also be useful to study Continuous-Time Dynamic Graph (CTDG) models and explainable graph neural networks to improve temporal tracking and make the model easier for compliance teams to interpret.

In the long run, the framework could evolve into a broader banking compliance platform instead of remaining only a detection system. That would allow features such as transaction freezing, adaptive risk-trigger handling, document verification, regulator-approved intervention workflows, and policy-based account restrictions to be connected more directly with core banking systems.

6.3 Learning Outcomes of the Project

Working on this project gave practical experience with heterogeneous graph-learning systems, federated learning, and real-time financial fraud detection. It helped build confidence in using Graph Neural Networks, temporal sequence modeling, distributed param-

eter synchronization, and privacy-preserving inference in an institutional setting. The work also strengthened skills in Apache Kafka streaming, PostgreSQL tuning, concurrent API testing, and full-stack integration with Next.js, Prisma ORM, and FastAPI.

Just as importantly, the project made the AML workflow more concrete. It clarified how STR generation, fraud monitoring, regulator-side review, and institutional escalation fit together in a real compliance pipeline. It also improved understanding of Precision–Recall evaluation, confusion-matrix interpretation, and scalability testing for systems that need to stay responsive while processing streaming transaction data.